

Триггеры

Обзор механизма работы триггеров. Триггерные функции



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Механизм триггеров позволяет выполнять определенные действия «в ответ» на определенные события. Триггер состоит из двух частей: собственно триггера (который определяет события) и триггерной функции (которая определяет действия). И триггер, и функция являются самостоятельными объектами БД.

Когда возникает событие, на которое «подписан» триггер, вызывается триггерная функция. Ей передается контекст вызова, чтобы можно было определить, какой именно триггер и в каких условиях вызвал функцию.

Триггерная функция — это обычная функция, которая написана с учетом некоторых соглашений:

- она пишется на любом языке, кроме чистого SQL;
- она не имеет параметров;
- она возвращает значение типа trigger (на самом деле это псевдотип, по факту возвращается запись, соответствующая строке таблицы).

Триггерная функция выполняется в той же транзакции, что и основная операция. Таким образом, если триггерная функция завершится с ошибкой, вся транзакция будет прервана.

<https://postgrespro.ru/docs/postgresql/16/trigger-definition>



Следующий материал взят из учебного курса «Язык SQL» Postgres Pro:
<https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-09.pdf>

Создание триггера



- Упрощенный синтаксис команды для создания триггера таков:
`CREATE TRIGGER имя_триггера { BEFORE | AFTER | INSTEAD OF
} { событие [OR ...] }
ON имя_таблицы
[FOR [EACH] { ROW | STATEMENT }]
EXECUTE PROCEDURE имя_функции (аргументы)`

- где «событие» – это одно из:

INSERT

UPDATE [OF имя_столбца [, ...]]

DELETE

TRUNCATE



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггеры и функции



Триггер

объект базы данных — список обрабатываемых событий
при возникновении события вызывается триггерная функция
и ей передается контекст

Триггерная функция

объект базы данных — код обработки события
выполняется в той же транзакции, что и основная операция
соглашение: функция не принимает параметры,
возвращает значение псевдотипа trigger (фактически record)
может использоваться в нескольких триггерах



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

СОБЫТИЯ



INSERT, UPDATE, DELETE

таблицы	before/after statement before/after row
представления	before/after statement instead of row

TRUNCATE

таблицы	before/after statement
---------	------------------------

Условие WHEN

устанавливает дополнительный фильтр



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггеры могут срабатывать на вставку (INSERT), обновление (UPDATE) или удаление (DELETE) строк в таблице или представлении, а также на опустошение (TRUNCATE) таблиц.

Триггер может срабатывать до выполнения действия (BEFORE), после него (AFTER), или вместо него (INSTEAD OF).

Триггер может срабатывать один раз для всей операции (FOR EACH STATEMENT), или каждый раз для каждой затронутой строки (FOR EACH ROW).

Не для любой комбинации этих условий можно создать триггер. Например, instead-of-триггеры можно определить только для представлений на уровне строк, а truncate-триггер можно определить только для таблиц и только на уровне оператора. Допустимые варианты перечислены на слайде.

Кроме того, можно сузить область действия триггера, указав дополнительное условие WHEN: если условие не выполняется — триггер не срабатывает.

<https://postgrespro.ru/docs/postgresql/16/sql-createtrigger>



Обзор механизма работы триггеров

Триггер является указанием, что база данных должна автоматически выполнить заданную функцию, всякий раз когда выполнен определённый тип операции.

Триггеры можно использовать с таблицами, с представлениями и с внешними таблицами.

Для обычных и сторонних таблиц можно определять триггеры, которые будут срабатывать до или после любой из команд INSERT, UPDATE или DELETE; либо один раз для каждой модифицируемой строки, либо один раз для оператора SQL.

Также триггеры могут срабатывать для операторов TRUNCATE (сторонние таблицы не поддерживают этот оператор вовсе).

Когда происходит событие триггера, для обработки этого события в установленный момент времени вызывается функция триггера.



Обзор механизма работы триггеров

Для представлений триггеры могут быть определены для выполнения вместо операций INSERT, UPDATE и DELETE.

Такие триггеры **INSTEAD OF** вызываются единожды для каждой строки, которая должна быть изменена в этом представлении. Именно функция триггера отвечает за то, чтобы произвести необходимые изменения в нижележащих базовых таблицах представления и должным образом возвращать изменённые строки, чтобы они появлялись в представлении.

Триггеры для представлений тоже могут быть определены так, что они будут выполняться единожды для всего оператора SQL, до или после операций INSERT, UPDATE или DELETE. Однако такие триггеры срабатывают, только если для представления определён триггер INSTEAD OF.

В противном случае все операторы, обращающиеся к представлению, должны быть переписаны в виде операторов, обращающихся к нижележащим базовым таблицам, и тогда будут срабатывать триггеры, установленные для этих таблиц.



Обзор механизма работы триггеров

Триггерная функция должна быть создана до триггера. Она должна быть объявлена без аргументов и возвращать тип `trigger`. (Триггерная функция получает данные на вход посредством специально переданной структуры `TriggerData`, а не в форме обычных аргументов.)

После создания триггерной функции создаётся триггер с помощью `CREATE TRIGGER`. Одна и та же триггерная функция может быть использована для нескольких триггеров.

Postgres Pro предлагает как [построчные триггеры](#), так и [операторные триггеры](#).

В случае [построчного триггера](#), триггерная функция вызывается один раз для каждой строки, затронутой оператором, запустившим триггер.

В противоположность этому, [операторный триггер](#) вызывается только один раз при выполнении соответствующего оператора, независимо от количества строк, которые он затрагивает.

В частности оператор, который вообще не затрагивает строк, все равно приведёт к срабатыванию операторного триггера.



Обзор механизма работы триггеров

Эти два типа триггеров иногда называют **триггерами уровня строк** и **триггерами уровня оператора** соответственно.

Триггеры **на TRUNCATE** могут быть определены только **на уровне оператора**.

Триггеры **для представлений**, срабатывающие до или после, могут быть определены только **на уровне оператора**.

Триггеры, срабатывающие **вместо команд INSERT, UPDATE или DELETE**, могут быть определены только **на уровне строк**.



Обзор механизма работы триггеров

Триггеры также классифицируются в соответствии с тем, срабатывают ли они до, после или вместо операции.

Они называются триггерами **BEFORE**, **AFTER** и **INSTEAD OF**, соответственно.

Триггеры **BEFORE** уровня оператора срабатывают до того, как оператор начинает делать что-либо, в то время как триггеры **AFTER** уровня оператора срабатывают в самом конце работы оператора.

Эти типы триггеров могут быть определены для таблиц или представлений.

Триггеры **BEFORE** уровня строки срабатывают непосредственно перед обработкой конкретной строки.

Триггеры **AFTER** уровня строки срабатывают в конце работы всего оператора (но до любого из триггеров **AFTER** уровня оператора). Эти типы триггеров могут определяться только для таблиц и сторонних таблиц.

Триггеры **INSTEAD OF** уровня строки могут определяться только для представлений и срабатывают для каждой строки, сразу после того как строка представления идентифицирована как нуждающаяся в обработке.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Before statement



Срабатывает

до операции

Возвращаемое значение

игнорируется

Контекст

TG-переменные

BEFORE STATEMENT

↓
выполнение операции



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Рассмотрим подробнее разные типы триггеров.

Триггер BEFORE STATEMENT срабатывает один раз для операции независимо от того, сколько строк будет затронуто (возможно, что и ни одной). Это происходит до того, как операция начала выполняться.

Возвращаемое значение триггерной функции игнорируется. Если в триггере возникает ошибка, операция отменяется.

Поскольку триггерная функция не имеет параметров, контекст вызова в PL/pgSQL передается ей с помощью predefined TG-переменных, таких, как:

- TG_WHEN = «BEFORE»,
- TG_LEVEL = «STATEMENT»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE»/«TRUNCATE»

и др. Триггерной функции можно также передать пользовательский контекст (аналог отсутствующих параметров) через переменную TG_ARGV, хотя зачастую лучше вместо одной «обобщенной» функции создать несколько «частных».

<https://postgrespro.ru/docs/postgresql/16/plpgsql-trigger>



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Before row



Срабатывает

перед действием со строкой
в процессе выполнения операции

Возвращаемое значение

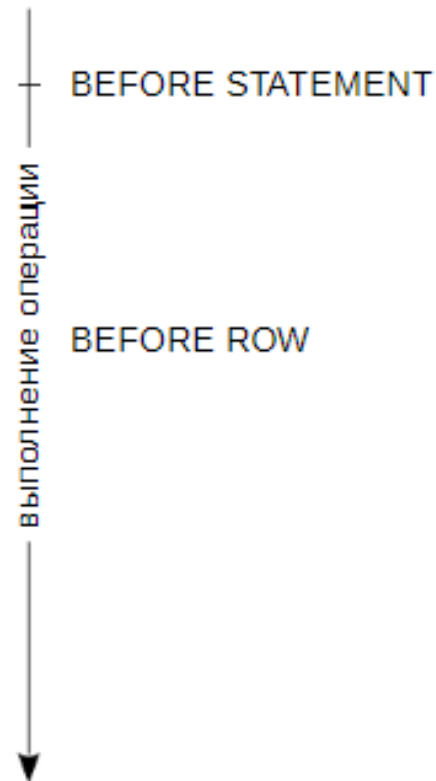
строка (возможно, измененная)
null отменяет действие

Контекст

OLD update, delete

NEW insert, update

TG-переменные



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггеры BEFORE ROW срабатывают каждый раз перед тем, как операция затронет строку. Это происходит непосредственно в процессе выполнения операции.

В качестве контекста триггерная функция получает переменные:

- OLD — старая строка (не определено для операции вставки),
- NEW — измененная строка (не определено для удаления),
- TG_WHEN = «BEFORE»,
- TG_LEVEL = «ROW»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE»

и др.



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Возврат неопределенного значения NULL воспринимается как отмена действия над данной строкой. Сама операция продолжит выполнение, но текущая строка не будет обработана и другие триггеры для этой строки не сработают.

Чтобы не вмешиваться в работу операции, триггер должен вернуть строку в том виде, в котором ее собирается изменить операция: NEW для вставки и обновления, любое значение (но не NULL) для удаления (обычно используют OLD).

Но триггерная функция может и изменить значение NEW, чтобы повлиять на результат операции — часто именно для этого такой триггер и создают.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Instead of row



Срабатывает

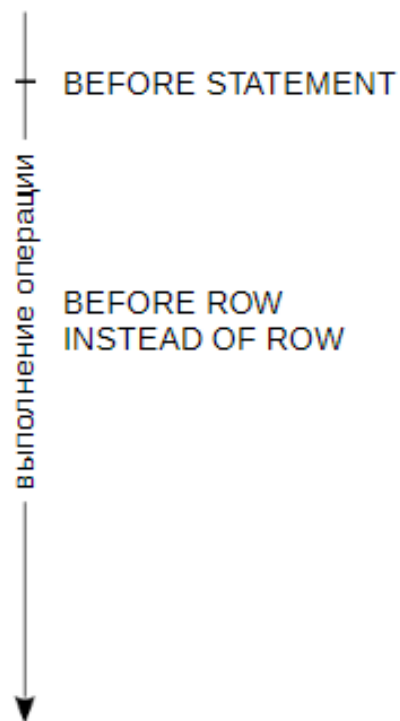
вместо действия со строкой
для представлений

Возвращаемое значение

строка (возможно, измененная) —
будет видна в RETURNING
null отменяет действие

Контекст

OLD update, delete
NEW insert, update
TG-переменные



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггеры `INSTEAD OF` очень похожи на триггеры `BEFORE`, но определяются только для представлений и срабатывают не до, а вместо операции.

В задачу таких триггеров обычно входит выполнение необходимых операций над базовыми таблицами представления. Также триггер может вернуть измененное значение `NEW` — именно оно будет видно при выполнении операции с указанием фразы `RETURNING`.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

After row



Срабатывает

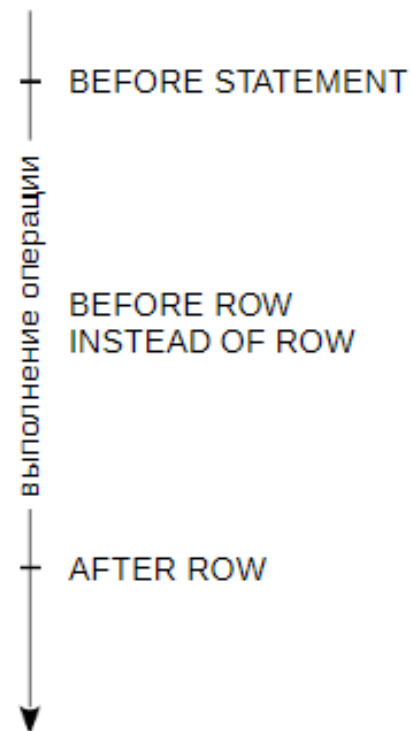
после выполнения операции
очередь из прошедших условия WHEN

Возвращаемое значение

игнорируется

Контекст

OLD, OLD TABLE update, delete
NEW, NEW TABLE insert, update
TG-переменные



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггеры AFTER ROW, как и BEFORE ROW, срабатывают для каждой затрагиваемой строки, но не сразу после действия над строкой, а после того, как выполнена вся операция - чтобы при обращении из этих триггеров к изменяющейся таблице результат не зависел от порядка обработки строк. Для этого события сначала помещаются в очередь и обрабатываются после окончания операции. Чем меньше событий попадет в очередь, тем меньше будет накладных расходов — поэтому именно в этом случае очень полезно использовать условие WHEN, чтобы отсеять заведомо ненужные строки.

Возвращаемое значение триггеров AFTER ROW игнорируется (поскольку операция уже выполнена).

Контекст триггерной функции составляют:

- OLD — старая строка (не определено для операции вставки),
- NEW — новое значение строки (не определено для удаления).



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Кроме этих переменных, начиная с версии 10 триггерная функция может получить доступ к специальным *переходным таблицам* (transition tables). Таблица, указанная при создании триггера как OLD TABLE, содержит старые значения строк, обработанных триггером, а таблица NEW TABLE — новые значения тех же строк.

Доступны и обычные TG-переменные, включая:

- TG_WHEN = «AFTER»,
- TG_LEVEL = «ROW»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE».



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

After statement



Срабатывает

после операции
(даже если не затронута ни одна строка)

Возвращаемое значение

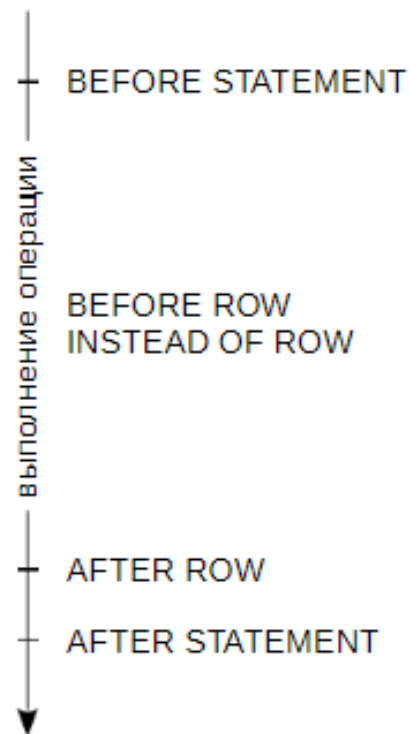
игнорируется

Контекст

OLD TABLE update, delete

NEW TABLE insert, update

TG-переменные



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггер AFTER STATEMENT срабатывает один раз после окончания операции и после всех триггеров AFTER ROW (независимо от того, сколько строк было затронуто).

Возвращаемое значение триггерной функции игнорируется.

Контекст вызова передается с помощью переходных таблиц.

Обращаясь к ним, триггерная функция может проанализировать все затронутые строки. Обычно переходные таблицы используются именно с триггерами AFTER STATEMENT, а не AFTER ROW.

Также определены обычные TG-переменные:

- TG_WHEN = «AFTER»,
- TG_LEVEL = «STATEMENT»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE»/«TRUNCATE»

и др.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Порядок вызова триггеров

Создадим «универсальную» триггерную функцию, которая описывает контекст, в котором она вызвана. Контекст передается в различных TG-переменных.

Затем создадим триггеры на различные события, и будем смотреть, какие триггеры и в каком порядке вызываются при выполнении операций.



Команда RAISE (для триггерной функции)

Команда RAISE предназначена для вывода сообщений и вызова ошибок.

Вариант синтаксиса:

```
RAISE [ level ] 'format' [, expression [, ... ]] [ USING option = expression [, ... ] ];
```

level задаёт уровень важности ошибки. Возможные значения: DEBUG, LOG, INFO, NOTICE, WARNING и EXCEPTION. По умолчанию - EXCEPTION. EXCEPTION вызывает ошибку, остальные значения level только генерируют сообщения с различными уровнями приоритета.

После level можно записать format (это должна быть простая символьная строка, не являющаяся выражением). Строка format задаёт текст сообщения. После format может следовать список необязательных аргументов выражений для вставки в текст сообщения.

Внутри строки format символ % заменяется на текстовое представление следующего необязательного значения аргумента. Чтобы вставить символ %, используйте %%.

В следующем примере символ % будет заменён на значение v_job_id:

```
RAISE NOTICE 'Вызов функции cs_create_job(%)', v_job_id; -- будет выведено: Вызов функции cs_create_job(v_job_id)
```

При помощи USING и последующих элементов option = expression можно добавить дополнительную информацию к отчёту об ошибке (все expression - строковые выражения).



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> CREATE OR REPLACE FUNCTION describe() RETURNS trigger
AS $$
DECLARE
    rec record;
    str text := '';
BEGIN
    IF TG_LEVEL = 'ROW' THEN
        CASE TG_OP
            WHEN 'DELETE' THEN rec := OLD; str := OLD::text;
            WHEN 'UPDATE' THEN rec := NEW; str := OLD || ' -> ' || NEW;
            WHEN 'INSERT' THEN rec := NEW; str := NEW::text;
        END CASE;
    END IF;
    RAISE NOTICE '% % % %: %',
        TG_TABLE_NAME, TG_WHEN, TG_OP, TG_LEVEL, str;
    RETURN rec;
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Таблица:

```
=> CREATE TABLE t(  
    id integer PRIMARY KEY,  
    s text  
);
```

CREATE TABLE

Триггеры на уровне оператора.

```
=> CREATE TRIGGER t_before_stmt  
BEFORE INSERT OR UPDATE OR DELETE -- события  
ON t                               -- таблица  
FOR EACH STATEMENT                -- уровень  
EXECUTE FUNCTION describe();      -- триггерная функция
```

CREATE TRIGGER

```
=> CREATE TRIGGER t_after_stmt  
AFTER INSERT OR UPDATE OR DELETE ON t  
FOR EACH STATEMENT EXECUTE FUNCTION describe();
```

CREATE TRIGGER



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

II на уровне строк:

```
=> CREATE TRIGGER t_before_row  
BEFORE INSERT OR UPDATE OR DELETE ON t  
FOR EACH ROW EXECUTE FUNCTION describe();
```

CREATE TRIGGER

```
=> CREATE TRIGGER t_after_row  
AFTER INSERT OR UPDATE OR DELETE ON t  
FOR EACH ROW EXECUTE FUNCTION describe();
```

CREATE TRIGGER

Пробуем вставку:

```
=> INSERT INTO t VALUES (1, 'aaa'), (2, 'bbb');
```

```
NOTICE:  t BEFORE INSERT STATEMENT:  
NOTICE:  t BEFORE INSERT ROW: (1,aaa)  
NOTICE:  t BEFORE INSERT ROW: (2,bbb)  
NOTICE:  t AFTER INSERT ROW: (1,aaa)  
NOTICE:  t AFTER INSERT ROW: (2,bbb)  
NOTICE:  t AFTER INSERT STATEMENT:  
INSERT 0 2
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Обновление:

```
=> UPDATE t SET s = 'ccc' WHERE id = 1;
```

```
NOTICE:  t BEFORE UPDATE STATEMENT:  
NOTICE:  t BEFORE UPDATE ROW: (1,aaa) -> (1,ccc)  
NOTICE:  t AFTER UPDATE ROW: (1,aaa) -> (1,ccc)  
NOTICE:  t AFTER UPDATE STATEMENT:  
UPDATE 1
```

Триггеры на уровне оператора срабатывают, даже если команда не обработала ни одной строки:

```
=> UPDATE t SET s = 'ddd' WHERE id = 0;
```

```
NOTICE:  t BEFORE UPDATE STATEMENT:  
NOTICE:  t AFTER UPDATE STATEMENT:  
UPDATE 0
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Тонкий момент: оператор INSERT с предложением ON CONFLICT приводит к тому, что срабатывают BEFORE-триггеры и на вставку, и на обновление:

```
=> INSERT INTO t VALUES (1,'ddd'), (3,'eee')
ON CONFLICT(id) DO UPDATE SET s = EXCLUDED.s;
```

```
NOTICE:  t BEFORE INSERT STATEMENT:
NOTICE:  t BEFORE UPDATE STATEMENT:
NOTICE:  t BEFORE INSERT ROW: (1,ddd)
NOTICE:  t BEFORE UPDATE ROW: (1,ccc) -> (1,ddd)
NOTICE:  t BEFORE INSERT ROW: (3,eee)
NOTICE:  t AFTER UPDATE ROW: (1,ccc) -> (1,ddd)
NOTICE:  t AFTER INSERT ROW: (3,eee)
NOTICE:  t AFTER UPDATE STATEMENT:
NOTICE:  t AFTER INSERT STATEMENT:
INSERT 0 2
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

II, наконец, удаление:

```
=> DELETE FROM t WHERE id = 2;
```

```
NOTICE:  t BEFORE DELETE STATEMENT:
NOTICE:  t BEFORE DELETE ROW: (2,bbb)
NOTICE:  t AFTER DELETE ROW: (2,bbb)
NOTICE:  t AFTER DELETE STATEMENT:
DELETE 1
```

Для появившегося в PostgreSQL 15 оператора MERGE специального триггера нет, работают триггеры на обновление, удаление и вставку:

```
=> MERGE INTO t
USING (VALUES (1, 'fff'), (3, 'ggg'), (4, 'hhh')) AS vals(id, s)
ON t.id = vals.id
WHEN MATCHED AND t.id = 1 THEN
    UPDATE SET s = vals.s
WHEN MATCHED THEN
    DELETE
WHEN NOT MATCHED THEN
    INSERT (id, s)
    VALUES (vals.id, vals.s);
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
NOTICE:  t BEFORE INSERT STATEMENT:
NOTICE:  t BEFORE UPDATE STATEMENT:
NOTICE:  t BEFORE DELETE STATEMENT:
NOTICE:  t BEFORE UPDATE ROW: (1,ddd) -> (1,fff)
NOTICE:  t BEFORE DELETE ROW: (3,eee)
NOTICE:  t BEFORE INSERT ROW: (4,hhh)
NOTICE:  t AFTER UPDATE ROW: (1,ddd) -> (1,fff)
NOTICE:  t AFTER DELETE ROW: (3,eee)
NOTICE:  t AFTER INSERT ROW: (4,hhh)
NOTICE:  t AFTER DELETE STATEMENT:
NOTICE:  t AFTER UPDATE STATEMENT:
NOTICE:  t AFTER INSERT STATEMENT:
MERGE 3
```

Переходные таблицы

Напишем триггерную функцию, показывающую содержимое переходных таблиц. Здесь мы используем имена `old_table` и `new_table`, которые будут объявлены при создании триггера.

Переходные таблицы «выглядят» настоящими, но не присутствуют в системном каталоге и располагаются в оперативной памяти (хотя и могут сбрасываться на диск при большом объеме).



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> CREATE OR REPLACE FUNCTION transition() RETURNS trigger
AS $$
DECLARE
    rec record;
BEGIN
    IF TG_OP = 'DELETE' OR TG_OP = 'UPDATE' THEN
        RAISE NOTICE 'Старое состояние: ';
        FOR rec IN SELECT * FROM old_table LOOP
            RAISE NOTICE '%', rec;
        END LOOP;
    END IF;
    IF TG_OP = 'UPDATE' OR TG_OP = 'INSERT' THEN
        RAISE NOTICE 'Новое состояние: ';
        FOR rec IN SELECT * FROM new_table LOOP
            RAISE NOTICE '%', rec;
        END LOOP;
    END IF;
    RETURN NULL;
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Создадим новую таблицу:

```
=> CREATE TABLE trans(  
    id integer PRIMARY KEY,  
    n integer  
);
```

CREATE TABLE

```
=> INSERT INTO trans VALUES (1,10), (2,20), (3,30);
```

INSERT 0 3

Чтобы при выполнении операции создавались переходные таблицы, необходимо указывать их имена при создании триггера:

```
=> CREATE TRIGGER t_after_upd_trans  
AFTER UPDATE ON trans -- только одно событие на один триггер  
REFERENCING  
    OLD TABLE AS old_table  
    NEW TABLE AS new_table -- можно и одну, не обязательно обе  
FOR EACH STATEMENT  
EXECUTE FUNCTION transition();
```

CREATE TRIGGER



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Проверим:

```
=> UPDATE trans SET n = n + 1 WHERE n <= 20;
```

```
NOTICE: Старое состояние:
```

```
NOTICE: (1,10)
```

```
NOTICE: (2,20)
```

```
NOTICE: Новое состояние:
```

```
NOTICE: (1,11)
```

```
NOTICE: (2,21)
```

```
UPDATE 2
```

Переходные таблицы содержат только те строки, которые были затронуты операцией.

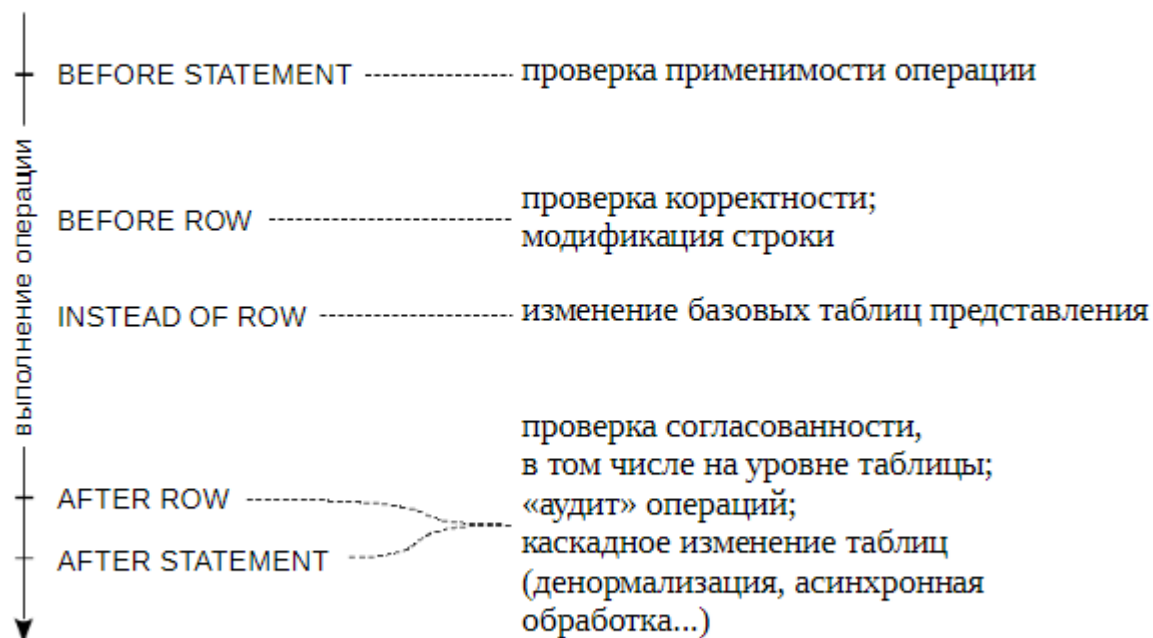
Для операций вставки и удаления переходные таблицы работают точно так же, но доступна будет только одна из таблиц: NEW TABLE или OLD TABLE соответственно.

Поскольку триггеры AFTER ROW срабатывают после выполнения всей операции, переходные таблицы можно использовать и в них. Но обычно это не имеет смысла.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Возможное использование



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Каково практическое применение триггеров?

BEFORE-триггеры можно использовать для проверки корректности операции и при необходимости вызывать ошибку.

Триггеры BEFORE ROW можно применять для модификации строки (например, заполнить пустое поле нужным значением). Это бывает удобно, чтобы не повторять логику заполнения «технических» полей в каждой операции, а также позволяет вмешаться в работу приложения, код которого недоступен для изменения.

Триггеры INSTEAD OF ROW применяют для того, чтобы отобразить операции над представлением в операции над базовыми таблицами.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггеры AFTER ROW и AFTER STATEMENT полезны в случаях, когда нужно знать точное состояние после операции (BEFORE-триггеры могут влиять на результат, так что на этом этапе ясности еще нет):

- для проверки согласованности операции;
- для «аудита», то есть записи изменений в отдельное хранилище;
- для каскадного изменения других таблиц (например, обновлять денормализованные данные при изменении базовых таблиц, или записывать изменения в очередь для последующей обработки вне данной транзакции).

Если операции затрагивают сразу много строк, то триггер AFTER STATEMENT с переходными таблицами может оказаться более эффективным решением, чем триггер AFTER ROW, поскольку позволяет обрабатывать все изменения пакетно, а не построчно.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

СЛОЖНОСТИ



Код вызывается неявно

сложно отследить логику выполнения

Правила видимости изменчивой триггерной функции

виден результат триггеров BEFORE ROW или INSTEAD OF ROW

Порядок вызова триггеров для одного события

триггеры обрабатывают в алфавитном порядке

Не предотвращается заикливание

триггер может вызвать срабатывание других триггеров

Можно нарушить ограничения целостности

например, исключив из обработки строки, которые должны удалиться



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Однако не стоит злоупотреблять триггерами. Триггеры срабатывают неявно, что сильно затрудняет понимание логики работы и крайне усложняет поддержку приложения. Попытки реализовать сложную логику на триггерах обычно заканчиваются плачевно.

В некоторых случаях вместо триггеров можно использовать вычисляемые поля (GENERATED ALWAYS AS ... STORED). Такое решение — если оно подходит — будет заведомо проще и прозрачнее.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Есть ряд тонкостей, связанных с триггерами, которые мы сознательно не рассматриваем подробно:

- правила видимости изменчивых (volatile) функций в триггерах BEFORE ROW и INSTEAD OF ROW (не стоит обращаться к таблице, полагаясь на порядок, в котором срабатывают триггеры);
- порядок вызова нескольких триггеров, обрабатывающих одно и то же событие (не стоит усугублять и без того неявное срабатывание триггеров завязкой на последовательность обработки);
- возможность заикливания в случае каскадного срабатывания других триггеров, которые, в свою очередь, могут приводить к новым срабатываниям данного триггера;
- возможность нарушить ограничения целостности (например, при исключении из обработки строки, которая удаляется условием ON DELETE CASCADE, может быть нарушена ссылочная целостность).

Если вы столкнулись с тем, что эти тонкости важны для вашего приложения — серьезно задумайтесь.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Примеры использования триггеров

Пример 1: сохранение истории изменения строк.

Пусть есть таблица, содержащая актуальные данные. Задача состоит в том, чтобы в отдельной таблице сохранять всю историю изменения строк основной таблицы.

Поддержку исторической таблицы можно было бы возложить на приложение, но тогда велика вероятность, что в каких-то случаях из-за ошибок история не будет сохраняться. Поэтому решим задачу с помощью триггера.

Основная таблица:

```
=> CREATE TABLE coins(  
    face_value numeric PRIMARY KEY,  
    name text  
);
```

CREATE TABLE



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Историческая таблица должна называться так же, как основная, но заканчиваться на «_history». Сначала создаем клон основной таблицы...

```
=> CREATE TABLE coins_history(LIKE coins);
```

CREATE TABLE

...и затем добавляем столбцы «действительно с» и «действительно по»:

```
=> ALTER TABLE coins_history  
    ADD start_date timestamp,  
    ADD end_date timestamp;
```

ALTER TABLE



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Одна триггерная функция будет вставлять новую историческую строку с открытым интервалом действия:

```
=> CREATE OR REPLACE FUNCTION history_insert() RETURNS trigger
AS $$
BEGIN
    EXECUTE format(
        'INSERT INTO %I SELECT ($1).*, current_timestamp, NULL',
        TG_TABLE_NAME || '_history'
    ) USING NEW;

    RETURN NEW;
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Другая функция будет закрывать интервал действия исторической строки:

```
=> CREATE OR REPLACE FUNCTION history_delete() RETURNS trigger
AS $$
BEGIN
    EXECUTE format(
        'UPDATE %I SET end_date = current_timestamp WHERE face_value = $1 AND end_date IS NULL',
        TG_TABLE_NAME || '_history'
    ) USING OLD.face_value;

    RETURN OLD;
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Теперь создадим триггеры. Важные моменты:

- Обновление трактуется как удаление и последующая вставка; здесь важен порядок, в котором сработают триггеры (по алфавиту).
- Current_timestamp возвращает время начала транзакции, поэтому при обновлении start_date одной строки будет равен end_date другой.
- Использование AFTER-триггеров позволяет избежать проблем с INSERT ... ON CONFLICT и потенциальными конфликтами с другими триггерами, которые могут существовать на основной таблице.

```
=> CREATE TRIGGER coins_history_insert  
AFTER INSERT OR UPDATE ON coins  
FOR EACH ROW EXECUTE FUNCTION history_insert();
```

CREATE TRIGGER

```
=> CREATE TRIGGER coins_history_delete  
AFTER UPDATE OR DELETE ON coins  
FOR EACH ROW EXECUTE FUNCTION history_delete();
```

CREATE TRIGGER



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Проверим работу триггеров.

```
=> INSERT INTO coins VALUES (0.25, 'Полушка'), (3, 'Алтын');
```

```
INSERT 0 2
```

```
=> UPDATE coins SET name = '3 копейки' WHERE face_value = 3;
```

```
UPDATE 1
```

```
=> INSERT INTO coins VALUES (5, '5 копеек');
```

```
INSERT 0 1
```

```
=> DELETE FROM coins WHERE face_value = 0.25;
```

```
DELETE 1
```

```
=> SELECT * FROM coins;
```

face_value	name
3	3 копейки
5	5 копеек

(2 rows)



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

В исторической таблице хранится вся история изменений:

```
=> SELECT * FROM coins_history ORDER BY face_value, start_date;
```

face_value	name	start_date	end_date
0.25	Полушка	2024-02-06 11:20:16.142265	2024-02-06 11:20:21.365151
3	Алтын	2024-02-06 11:20:16.142265	2024-02-06 11:20:18.204276
3	3 копейки	2024-02-06 11:20:18.204276	
5	5 копеек	2024-02-06 11:20:19.287525	

(4 rows)

И теперь по ней можно восстановить состояние на любой момент времени (это немного напоминает работу механизма MVCC). Например, на самое начало:

```
=> \set d '2024-02-06 11:20:16.185224+03'
```

```
=> SELECT face_value, name  
FROM coins_history  
WHERE start_date <= :d AND (end_date IS NULL OR :d < end_date)  
ORDER BY face_value;
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> SELECT face_value, name
FROM coins_history
WHERE start_date <= :'d' AND (end_date IS NULL OR :'d' < end_date)
ORDER BY face_value;
```

face_value	name
0.25	Полушка
3	Алтын

(2 rows)



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Примеры использования триггеров

Пример 2: обновляемое представление.

Пусть имеются две таблицы: аэропорты и рейсы:

```
=> CREATE TABLE airports(  
    code char(3) PRIMARY KEY,  
    name text NOT NULL  
);
```

CREATE TABLE

```
=> INSERT INTO airports VALUES  
    ('SVO', 'Москва. Шереметьево'),  
    ('LED', 'Санкт-Петербург. Пулково'),  
    ('TOF', 'Томск. Богашево');
```

INSERT 0 3

```
=> CREATE TABLE flights(  
    id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    airport_from char(3) NOT NULL REFERENCES airports(code),  
    airport_to char(3) NOT NULL REFERENCES airports(code),  
    UNIQUE (airport_from, airport_to)  
);
```

CREATE TABLE

```
=> INSERT INTO flights(airport_from, airport_to) VALUES  
    ('SVO', 'LED');
```

INSERT 0 1



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Для удобства можно определить представление:

```
=> CREATE VIEW flights_v AS
SELECT id,
       (SELECT name
        FROM airports
        WHERE code = airport_from) airport_from,
       (SELECT name
        FROM airports
        WHERE code = airport_to) airport_to
FROM flights;
```

CREATE VIEW

```
=> SELECT * FROM flights_v;
```

id	airport_from	airport_to
1	Москва. Шереметьево	Санкт-Петербург. Пулково

(1 row)



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Но такое представление не допускает изменений. Например, не получится изменить пункт назначения таким образом:

```
=> UPDATE flights_v  
SET airport_to = 'Томск. Богашево'  
WHERE id = 1;
```

```
ERROR:  cannot update column "airport_to" of view "flights_v"  
DETAIL:  View columns that are not columns of their base relation are not updatable.
```

Однако мы можем определить триггер. Триггерная функция может выглядеть, например, так (для краткости обрабатываем только аэропорт назначения, но не составит труда добавить и аэропорт вылета):



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> CREATE OR REPLACE FUNCTION flights_v_update() RETURNS trigger
AS $$
DECLARE
    code_to char(3);
BEGIN
    BEGIN
        SELECT code INTO STRICT code_to
        FROM airports
        WHERE name = NEW.airport_to;
    EXCEPTION
        WHEN no_data_found THEN
            RAISE EXCEPTION 'Аэропорт "%" отсутствует', NEW.airport_to;
    END;
    UPDATE flights
    SET airport_to = code_to
    WHERE id = OLD.id; -- изменение id игнорируем
    RETURN NEW;
END
$$ LANGUAGE plpgsql;
```

CREATE FUNCTION



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

И сам триггер:

```
=> CREATE TRIGGER flights_v_upd_trigger  
INSTEAD OF UPDATE ON flights_v  
FOR EACH ROW EXECUTE FUNCTION flights_v_update();
```

CREATE TRIGGER

Проверим:

```
=> UPDATE flights_v  
SET airport_to = 'Томск. Богашево'  
WHERE id = 1;
```

UPDATE 1

```
=> SELECT * FROM flights_v;
```

id	airport_from	airport_to
1	Москва. Шереметьево	Томск. Богашево

(1 row)



Что такое **SELECT INTO** (из функции на следующем слайде)

Выполнение запроса, возвращающего одну строку

Результат SQL-команды, возвращающей одну строку (возможно из нескольких столбцов), может быть присвоен переменной типа record, переменной строкового типа или списку скалярных переменных. Для этого нужно к основной команде SQL добавить предложение INTO. Так, например:

```
SELECT select_expressions INTO [STRICT] target FROM ...;  
INSERT ... RETURNING expressions INTO [STRICT] target;  
UPDATE ... RETURNING expressions INTO [STRICT] target;  
DELETE ... RETURNING expressions INTO [STRICT] target;
```

где target может быть переменной типа record, строковой переменной или разделённым запятыми списком скалярных переменных, полей записи/строки.

STRICT означает, что запрос должен вернуть ровно одну строку (в target вставляется ровно 1 строка).



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> CREATE OR REPLACE FUNCTION flights_v_update() RETURNS trigger
AS $$
DECLARE
    code_to char(3);
BEGIN
    BEGIN
        SELECT code INTO STRICT code_to
        FROM airports
        WHERE name = NEW.airport_to;
    EXCEPTION
        WHEN no_data_found THEN
            RAISE EXCEPTION 'Аэропорт "%" отсутствует', NEW.airport_to;
    END;
    UPDATE flights
    SET airport_to = code_to
    WHERE id = OLD.id; -- изменение id игнорируем
    RETURN NEW;
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

И сам триггер:

```
=> CREATE TRIGGER flights_v_upd_trigger  
INSTEAD OF UPDATE ON flights_v  
FOR EACH ROW EXECUTE FUNCTION flights_v_update();
```

CREATE TRIGGER

Проверим:

```
=> UPDATE flights_v  
SET airport_to = 'Томск. Богашево'  
WHERE id = 1;
```

UPDATE 1

```
=> SELECT * FROM flights_v;
```

id	airport_from	airport_to
1	Москва. Шереметьево	Томск. Богашево

(1 row)



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Попытка изменить аэропорт на отсутствующий в таблице:

```
=> UPDATE flights_v  
SET airport_to = 'Южно-Сахалинск. Хомутово'  
WHERE id = 1;
```

```
ERROR:  Аэропорт "Южно-Сахалинск. Хомутово" отсутствует  
CONTEXT:  PL/pgSQL function flights_v_update() line 11 at RAISE
```



Пример использования триггера INSTEAD OF для обновления данных в таблицах через представление

Представления (views) обычно используются для того, чтобы инкапсулировать сложные запросы и предоставить пользователям упрощенный доступ к данным. Однако, представление само по себе не хранит данные и не поддерживает прямые изменения (вставка, обновление, удаление).

Когда нужно позволить пользователям изменять данные через представление (например, вставлять или обновлять записи в таблицах, которые оно оборачивает), триггер INSTEAD OF позволяет перенаправить эти операции на соответствующие таблицы.

Пусть имеется представление, которое объединяет таблицы employees и departments. Если нужно разрешить пользователям вставлять или обновлять данные о сотрудниках через это представление, можно использовать триггер INSTEAD OF, чтобы вставлять или обновлять данные непосредственно в таблицы employees и departments при изменении данных через представление.



Пример использования триггера INSTEAD OF для обновления данных в таблицах через представление

```
CREATE VIEW employee_view AS
SELECT e.employee_id, e.name, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id;
```

```
CREATE OR REPLACE FUNCTION update_employee_view() RETURNS trigger AS $$
BEGIN
    -- Логика для обновление таблиц employees и departments
    UPDATE employees
    SET name = NEW.name
    WHERE employee_id = NEW.employee_id;

    UPDATE departments
    SET department_name = NEW.department_name
    WHERE department_id = NEW.department_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER employee_view_trigger
INSTEAD OF UPDATE ON employee_view
FOR EACH ROW EXECUTE FUNCTION update_employee_view();
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Практика



1. Создайте триггер, обрабатывающий обновление поля `onhand_qty` представления `catalog_v`.

Проверьте, что в «Каталоге» появилась возможность заказывать книги.

2. Обеспечьте выполнение требования согласованности: количество книг на складе не может быть отрицательным (нельзя купить книгу, которой нет в наличии).

Внимательно проверьте правильность реализации, учитывая, что с приложением могут одновременно работать несколько пользователей.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

2. Может показаться, что достаточно создать AFTER-триггер на таблице operations, подсчитывающий сумму qty_change. Однако на уровне изоляции Read Committed, с которым работает приложение «Книжный магазин», нам придется блокировать таблицу operations в эксклюзивном режиме — иначе возможны сценарии, при которых такая проверка не сработает.

Лучше поступить следующим образом: добавить в таблицу books поле onhand_qty и создать триггер, изменяющий это поле при изменении таблицы operations (то есть, фактически, выполнить денормализацию данных). На поле onhand_qty теперь можно наложить ограничение CHECK, реализующее требование согласованности. А функция onhand_qty(), которую мы создавали ранее, больше не нужна.

Особое внимание надо уделить начальной установке значения, учитывая, что одновременно с выполнением наших операций в системе могут работать пользователи.



Для решения задачи создайте базу данных, см.:

https://edu.postgrespro.ru/16/dev1-16/dev1_07_bstore_schema.html

1. Схема и путь поиска

```
=> CREATE DATABASE bookstore;
```

```
CREATE DATABASE
```

```
=> \c bookstore
```

```
You are now connected to database "bookstore" as user "student".
```

```
=> CREATE SCHEMA bookstore;
```

```
CREATE SCHEMA
```

```
=> ALTER DATABASE bookstore SET search_path = bookstore, public;
```

```
ALTER DATABASE
```



Для решения задачи создайте базу данных, см.:

https://edu.postgrespro.ru/16/dev1-16/dev1_07_bstore_schema.html

2. Таблицы

Авторы:

```
=> CREATE TABLE authors(  
    author_id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    last_name text NOT NULL,  
    first_name text NOT NULL,  
    middle_name text  
);
```

CREATE TABLE

Книги:

```
=> CREATE TABLE books(  
    book_id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    title text NOT NULL  
);
```

CREATE TABLE



Для решения задачи создайте базу данных, см.:

https://edu.postgrespro.ru/16/dev1-16/dev1_07_bstore_schema.html

Авторство:

```
=> CREATE TABLE authorship(  
    book_id integer REFERENCES books,  
    author_id integer REFERENCES authors,  
    seq_num integer NOT NULL,  
    PRIMARY KEY (book_id,author_id)  
);
```

CREATE TABLE

Операции:

```
=> CREATE TABLE operations(  
    operation_id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    book_id integer NOT NULL REFERENCES books,  
    qty_change integer NOT NULL,  
    date_created date NOT NULL DEFAULT current_date  
);
```

CREATE TABLE



Для решения задачи создайте базу данных, см.:

https://edu.postgrespro.ru/16/dev1-16/dev1_07_bstore_schema.html

3. Данные

Авторы:

```
=> INSERT INTO authors(last_name, first_name, middle_name)
VALUES
    ('Пушкин', 'Александр', 'Сергеевич'),
    ('Тургенев', 'Иван', 'Сергеевич'),
    ('Стругацкий', 'Борис', 'Натанович'),
    ('Стругацкий', 'Аркадий', 'Натанович'),
    ('Толстой', 'Лев', 'Николаевич'),
    ('Свифт', 'Джонатан', NULL);
```

INSERT 0 6

Книги:

```
=> INSERT INTO books(title)
VALUES
    ('Сказка о царе Салтане'),
    ('Муму'),
    ('Трудно быть богом'),
    ('Война и мир'),
    ('Путешествия в некоторые удаленные страны мира в четырех частях: сочинение Лемюэля Гулливера, сначала хирурга, а затем капитана не
    скольких кораблей'),
```

INSERT 0 6



Для решения задачи создайте базу данных, см.:

https://edu.postgrespro.ru/16/dev1-16/dev1_07_bstore_schema.html

Авторство:

```
=> INSERT INTO authorship(book_id, author_id, seq_num)
```

```
VALUES
```

```
    (1, 1, 1),
```

```
    (2, 2, 1),
```

```
    (3, 3, 2),
```

```
    (3, 4, 1),
```

```
    (4, 5, 1),
```

```
    (5, 6, 1),
```

```
    (6, 1, 1),
```

```
    (6, 5, 2),
```

```
    (6, 2, 3);
```

```
INSERT 0 9
```

Операции:

```
=> INSERT INTO operations(book_id, qty_change)
```

```
VALUES
```

```
    (1, 10),
```

```
    (1, 10),
```

```
    (1, -1);
```

```
INSERT 0 3
```



Для решения задачи создайте базу данных, см.:

https://edu.postgrespro.ru/16/dev1-16/dev1_07_bstore_schema.html

4. Представления

Представление для авторов:

```
=> CREATE VIEW authors_v AS
SELECT a.author_id,
       a.last_name || ' ' ||
       a.first_name ||
       coalesce(' ' || nullif(a.middle_name, ''), '') AS display_name
FROM   authors a;
```

CREATE VIEW

Представление для каталога:

```
=> CREATE VIEW catalog_v AS
SELECT b.book_id,
       b.title AS display_name
FROM   books b;
```

CREATE VIEW



Для решения задачи создайте базу данных, см.:

https://edu.postgrespro.ru/16/dev1-16/dev1_07_bstore_schema.html

Представление для операций:

```
=> CREATE VIEW operations_v AS
SELECT book_id,
       CASE
         WHEN qty_change > 0 THEN 'Поступление'
         ELSE 'Покупка'
       END op_type,
       abs(qty_change) qty_change,
       to_char(date_created, 'DD.MM.YYYY') date_created
FROM   operations
ORDER BY operation_id;
```

CREATE VIEW



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

1. Триггер для обновления каталога

```
=> CREATE OR REPLACE FUNCTION update_catalog() RETURNS trigger
AS $$
BEGIN
    INSERT INTO operations(book_id, qty_change) VALUES
        (OLD.book_id, NEW.onhand_qty - coalesce(OLD.onhand_qty,0));
    RETURN NEW;
END
$$ VOLATILE LANGUAGE plpgsql;
```

CREATE FUNCTION

```
=> CREATE TRIGGER update_catalog_trigger
INSTEAD OF UPDATE ON catalog_v
FOR EACH ROW
EXECUTE FUNCTION update_catalog();
```

CREATE TRIGGER



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

2. Проверка количества книг

Добавляем к таблице книг поле наличного количества. (До версии 11 важно было учитывать, что указание предложения DEFAULT вызывало перезапись всех строк таблицы, удерживая блокировку.)

```
=> ALTER TABLE books ADD COLUMN onhand_qty integer;
```

```
ALTER TABLE
```

Триггерная функция для AFTER-триггера на вставку для обновления количества (предполагаем, что поле onhand_qty не может быть пустым):

```
=> CREATE OR REPLACE FUNCTION update_onhand_qty() RETURNS trigger
AS $$
BEGIN
    UPDATE books
    SET onhand_qty = onhand_qty + NEW.qty_change
    WHERE book_id = NEW.book_id;
    RETURN NULL;
END
$$ VOLATILE LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Дальше все происходит внутри транзакции.

=> **BEGIN;**

BEGIN

Блокируем операции на время транзакции:

=> **LOCK TABLE** operations;

LOCK TABLE

Начальное заполнение:

```
=> UPDATE books b
SET onhand_qty = (
    SELECT coalesce(sum(qty_change), 0)
    FROM operations o
    WHERE o.book_id = b.book_id
);
```

UPDATE 6

Теперь, когда поле заполнено, задаем ограничения:



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> ALTER TABLE books ALTER COLUMN onhand_qty SET DEFAULT 0;
```

```
ALTER TABLE
```

```
=> ALTER TABLE books ALTER COLUMN onhand_qty SET NOT NULL;
```

```
ALTER TABLE
```

```
=> ALTER TABLE books ADD CHECK(onhand_qty >= 0);
```

```
ALTER TABLE
```

Создаем триггер:

```
=> CREATE TRIGGER update_onhand_qty_trigger  
AFTER INSERT ON operations  
FOR EACH ROW  
EXECUTE FUNCTION update_onhand_qty();
```

```
CREATE TRIGGER
```

Готово.

```
=> COMMIT;
```

```
COMMIT
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Теперь books.onhand_qty обновляется, но представление catalog_v по-прежнему вызывает функцию для подсчета количества. Хотя в исходном запросе обращение к функции синтаксически не отличается от обращения к полю, запрос был запомнен в другом виде:

=> \d+ catalog_v

View "bookstore.catalog_v"							
Column	Type	Collation	Nullable	Default	Storage	Description	
book_id	integer				plain		
title	text				extended		
onhand_qty	integer				plain		
display_name	text				extended		
authors	text				extended		

View definition:

```
SELECT book_id,
       title,
       onhand_qty(b.*) AS onhand_qty,
       book_name(book_id, title) AS display_name,
       authors(b.*) AS authors
FROM books b
ORDER BY (book_name(book_id, title));
```

Triggers:

```
update_catalog_trigger INSTEAD OF UPDATE ON catalog_v FOR EACH ROW EXECUTE FUNCTION
update_catalog()
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Поэтому пересоздадим представление:

```
=> CREATE OR REPLACE VIEW catalog_v AS
SELECT b.book_id,
       b.title,
       b.onhand_qty,
       book_name(b.book_id, b.title) AS display_name,
       b.authors
FROM   books b
ORDER BY display_name;
```

CREATE VIEW

Теперь функцию можно удалить.

```
=> DROP FUNCTION onhand_qty(books);
```

DROP FUNCTION



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Небольшая проверка:

```
=> SELECT * FROM catalog_v WHERE book_id = 1 \gx
```

```
-[ RECORD 1 ]+-----  
book_id      | 1  
title        | Сказка о царе Салтане  
onhand_qty   | 19  
display_name | Сказка о царе Салтане. Пушкин А. С.  
authors      | Пушкин Александр Сергеевич
```

```
=> INSERT INTO operations(book_id, qty_change) VALUES (1,+10);
```

```
INSERT 0 1
```

```
=> SELECT * FROM catalog_v WHERE book_id = 1 \gx
```

```
-[ RECORD 1 ]+-----  
book_id      | 1  
title        | Сказка о царе Салтане  
onhand_qty   | 29  
display_name | Сказка о царе Салтане. Пушкин А. С.  
authors      | Пушкин Александр Сергеевич
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Некорректные операции обрываются:

```
=> INSERT INTO operations(book_id, qty_change) VALUES (1,-100);
```

```
ERROR: new row for relation "books" violates check constraint "books_onhand_qty_check"
```

```
DETAIL: Failing row contains (1, Сказка о царе Салтане, -71).
```

```
CONTEXT: SQL statement "UPDATE books
```

```
    SET onhand_qty = onhand_qty + NEW.qty_change
```

```
    WHERE book_id = NEW.book_id"
```

```
PL/pgSQL function update_onhand_qty() line 3 at SQL statement
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Практика



1. Напишите триггер, увеличивающий счетчик (поле version) на единицу при каждом изменении строки. При вставке новой строки счетчик должен устанавливаться в единицу. Проверьте правильность работы.
2. Даны таблицы заказов (orders) и строк заказов (lines). Требуется выполнить денормализацию: автоматически обновлять сумму заказа в таблице orders при изменении строк в заказе.
Создайте необходимые триггеры с использованием переходных таблиц для минимизации операций обновления.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

2. Для создания таблиц используйте команды:

```
CREATE TABLE orders (  
    id int PRIMARY KEY,  
    total_amount numeric(20,2) NOT NULL DEFAULT 0  
);
```

```
CREATE TABLE lines (  
    id int PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    order_id int NOT NULL REFERENCES orders(id),  
    amount numeric(20,2) NOT NULL  
);
```

Столбец `orders.total_amount` должен автоматически вычисляться как сумма значений столбца `lines.amount` всех строк, относящихся к соответствующему заказу.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

1. Счетчик номера версии

```
=> CREATE DATABASE plpgsql_triggers;
```

```
CREATE DATABASE
```

```
=> \c plpgsql_triggers
```

```
You are now connected to database "plpgsql_triggers" as user "student".
```

Таблица:

```
=> CREATE TABLE t(  
    id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    s text,  
    version integer  
);
```

```
CREATE TABLE
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Триггерная функция:

```
=> CREATE OR REPLACE FUNCTION inc_version() RETURNS trigger
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        NEW.version := 1;
    ELSE
        NEW.version := OLD.version + 1;
    END IF;
    RETURN NEW;
END
$$ LANGUAGE plpgsql;
```

CREATE FUNCTION

Триггер:

```
=> CREATE TRIGGER t_inc_version
BEFORE INSERT OR UPDATE ON t
FOR EACH ROW EXECUTE FUNCTION inc_version();
```

CREATE TRIGGER



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Проверяем:

```
=> INSERT INTO t(s) VALUES ('Раз');
```

```
INSERT 0 1
```

```
=> SELECT * FROM t;
```

id	s	version
1	Раз	1

(1 row)

Явное указание version игнорируется:

```
=> INSERT INTO t(s,version) VALUES ('Два',42);
```

```
INSERT 0 1
```

```
=> SELECT * FROM t;
```

id	s	version
1	Раз	1
2	Два	1

(2 rows)



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Изменение:

```
=> UPDATE t SET s = lower(s) WHERE id = 1;
```

UPDATE 1

```
=> SELECT * FROM t;
```

id	s	version
2	Два	1
1	раз	2

(2 rows)

Явное указание также игнорируется:

```
=> UPDATE t SET s = lower(s), version = 42 WHERE id = 2;
```

UPDATE 1

```
=> SELECT * FROM t;
```

id	s	version
1	раз	2
2	два	2

(2 rows)



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

2. Автоматическое вычисление общей суммы заказов

Создаем таблицы упрощенной структуры, достаточной для демонстрации:

```
=> CREATE TABLE orders (  
    id integer PRIMARY KEY,  
    total_amount numeric(20,2) NOT NULL DEFAULT 0  
);
```

CREATE TABLE

```
=> CREATE TABLE lines (  
    id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    order_id integer NOT NULL REFERENCES orders(id),  
    amount numeric(20,2) NOT NULL  
);
```

CREATE TABLE



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Создаем триггерную функцию и триггер для обработки вставки:

```
=> CREATE FUNCTION total_amount_ins() RETURNS trigger
AS $$
BEGIN
    WITH l(order_id, total_amount) AS (
        SELECT order_id, sum(amount)
        FROM new_table
        GROUP BY order_id
    )
    UPDATE orders o
    SET total_amount = o.total_amount + l.total_amount
    FROM l
    WHERE o.id = l.order_id;
    RETURN NULL;
END
$$ LANGUAGE plpgsql;
```

CREATE FUNCTION



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Предложение FROM в команде UPDATE позволяет соединить orders с подзапросом по переходной таблице и использовать столбцы подзапроса для вычисления значения.

```
=> CREATE TRIGGER lines_total_amount_ins
AFTER INSERT ON lines
REFERENCING
    NEW TABLE AS new_table
FOR EACH STATEMENT
EXECUTE FUNCTION total_amount_ins();

CREATE TRIGGER
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Функция и триггер для обработки обновления:

```
=> CREATE FUNCTION total_amount_upd() RETURNS trigger
AS $$
BEGIN
    WITH l_tmp(order_id, amount) AS (
        SELECT order_id, amount FROM new_table
        UNION ALL
        SELECT order_id, -amount FROM old_table
    ), l(order_id, total_amount) AS (
        SELECT order_id, sum(amount)
        FROM l_tmp
        GROUP BY order_id
        HAVING sum(amount) <> 0
    )
    UPDATE orders o
    SET total_amount = o.total_amount + l.total_amount
    FROM l
    WHERE o.id = l.order_id;
    RETURN NULL;
END
$$ LANGUAGE plpgsql;
```

CREATE FUNCTION



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Условие HAVING позволяет пропускать изменения, не влияющие на общую сумму заказа.

```
=> CREATE TRIGGER lines_total_amount_upd
AFTER UPDATE ON lines
REFERENCING
    OLD TABLE AS old_table
    NEW TABLE AS new_table
FOR EACH STATEMENT
EXECUTE FUNCTION total_amount_upd();

CREATE TRIGGER
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Функция и триггер для обработки удаления:

```
=> CREATE FUNCTION total_amount_del() RETURNS trigger
AS $$
BEGIN
    WITH l(order_id, total_amount) AS (
        SELECT order_id, -sum(amount)
        FROM old_table
        GROUP BY order_id
    )
    UPDATE orders o
    SET total_amount = o.total_amount + l.total_amount
    FROM l
    WHERE o.id = l.order_id;
    RETURN NULL;
END
$$ LANGUAGE plpgsql;

CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> CREATE TRIGGER lines_total_amount_del  
AFTER DELETE ON lines  
REFERENCING  
    OLD TABLE AS old_table  
FOR EACH STATEMENT  
EXECUTE FUNCTION total_amount_del();  
  
CREATE TRIGGER
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Остался неохваченным оператор TRUNCATE. Однако триггер для этого оператора не может использовать переходные таблицы. Но мы знаем, что после выполнения TRUNCATE в lines не останется строк, значит можно обнулить суммы всех заказов.

```
=> CREATE FUNCTION total_amount_truncate() RETURNS trigger
AS $$
BEGIN
    UPDATE orders SET total_amount = 0;
    RETURN NULL;
END
$$ LANGUAGE plpgsql;
```

CREATE FUNCTION

```
=> CREATE TRIGGER lines_total_amount_truncate
AFTER TRUNCATE ON lines
FOR EACH STATEMENT
EXECUTE FUNCTION total_amount_truncate();
```

CREATE TRIGGER

Дополнительно нужно запретить изменять значение total_amount вручную, но эта задача решается не триггерами.

Проверяем работу.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Проверяем работу.

Добавили два новых заказа без строк:

```
=> INSERT INTO orders VALUES (1), (2);
```

```
INSERT 0 2
```

```
=> SELECT * FROM orders ORDER BY id;
```

id	total_amount
1	0.00
2	0.00

(2 rows)

Добавили строки в заказы:

```
=> INSERT INTO lines (order_id, amount) VALUES  
    (1,100), (1,100), (2,500), (2,500);
```

```
INSERT 0 4
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> SELECT * FROM lines;
```

id	order_id	amount
1	1	100.00
2	1	100.00
3	2	500.00
4	2	500.00

(4 rows)

```
=> SELECT * FROM orders ORDER BY id;
```

id	total_amount
1	200.00
2	1000.00

(2 rows)

Удвоили суммы всех строк всех заказов:

```
=> UPDATE lines SET amount = amount * 2;
```

UPDATE 4



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

```
=> SELECT * FROM orders ORDER BY id;
```

id	total_amount
1	400.00
2	2000.00

(2 rows)

Удалим одну строку первого заказа:

```
=> DELETE FROM lines WHERE id = 1;
```

DELETE 1

```
=> SELECT * FROM orders ORDER BY id;
```

id	total_amount
1	200.00
2	2000.00

(2 rows)



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Опустошим таблицу строк:

```
=> TRUNCATE lines;
```

```
TRUNCATE TABLE
```

```
=> SELECT * FROM orders ORDER BY id;
```

id	total_amount
1	0.00
2	0.00

(2 rows)



Другой пример: (материал взят из учебного курса «Язык SQL» Postgres Pro:
<https://edu.postgrespro.ru/sqlprimer/sqlprimer-2019-msu-09.pdf>)

Подготовительные операции (1)



- Создадим две копии таблицы «Самолеты» (aircrafts).
- Первая будет предназначена для хранения данных, взятых из таблицы-прототипа, а вторая будет использоваться в качестве журнальной таблицы.

```
CREATE TABLE aircrafts_tmp AS  
  SELECT * FROM aircrafts WITH NO DATA;
```

```
ALTER TABLE aircrafts_tmp  
  ADD PRIMARY KEY ( aircraft_code );
```

```
ALTER TABLE aircrafts_tmp  
  ADD UNIQUE ( model );
```

данные не
копируем

Ограничения не создаются при
копировании таблицы



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Подготовительные операции (2)



```
CREATE TABLE aircrafts_log AS  
    SELECT * FROM aircrafts WITH NO DATA;  
ALTER TABLE aircrafts_log  
    ADD COLUMN when_add timestamp;  
ALTER TABLE aircrafts_log  
    ADD COLUMN operation text;
```

данные не
копируем

когда
выполнена
операция

INSERT,
UPDATE или
DELETE



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Пример триггерной функции и триггера



Задача: скопировать в таблицу `aircrafts_tmp` все данные из таблицы `aircrafts`, фиксируя все изменения в журнале изменений.

```
SET search_path = bookings;

CREATE OR REPLACE FUNCTION log_aircrafts()
RETURNS trigger AS
$$
BEGIN
    INSERT INTO aircrafts_log ( aircraft_code, model, range,
                               when_add, operation )
    VALUES ( NEW.aircraft_code, NEW.model, NEW.range,
              CURRENT_TIMESTAMP, 'INSERT' );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS aircrafts_log ON aircrafts_tmp;

CREATE TRIGGER aircrafts_log AFTER INSERT ON aircrafts_tmp
FOR EACH ROW EXECUTE PROCEDURE log_aircrafts();
```

Язык PL/pgSQL

Сохраним в файле `log_aircrafts.sql`



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_17_plpgsql_triggers.html

Создаем триггерную функцию и триггер



```
psql -d demo -f log_aircrafts.sql -U postgres
```

```
SET
```

```
CREATE FUNCTION
```

```
DROP TRIGGER
```

```
CREATE TRIGGER
```

```
INSERT INTO aircrafts_tmp SELECT * FROM aircrafts;
```

```
INSERT 0 9
```

```
SELECT * FROM aircrafts_log;
```

aircraft_code	model	range	when_add	operation
773	Boeing 777-300	11100	2018-10-10	INSERT
763	Boeing 767-300	7900	2018-10-10	INSERT
...				

(9 строк)



Операции над триггерами

Триггеры в PostgreSQL поддерживают следующие операции DML:

Create trigger - используется для создания триггера

Drop trigger - используется для удаления триггера

Alter trigger - используется для изменения имени существующего триггера

Disable trigger - используется для отключения заданного или всех триггеров, связанных с таблицей

Enable trigger - используется для включения конкретного или всех триггеров, связанных с таблицей



Изменение триггера

Чтобы изменить свойства триггера, используйте CREATE OR REPLACE TRIGGER, указав имя существующей триггерной функции и связанную таблицу. Остальные свойства можно менять так, как это нужно для выполнения задачи.

ALTER TRIGGER изменяет определение триггера.

Синтаксис

ALTER TRIGGER имя ON имя_таблицы RENAME TO новое_имя

ALTER TRIGGER имя ON имя_таблицы [NO] DEPENDS ON EXTENSION имя_расширения

Предложение RENAME переименовывает данный триггер, не затрагивая его определение.

Предложение DEPENDS ON EXTENSION помечает триггер как зависимый от расширения, так что при удалении расширения будет автоматически удаляться и триггер.

Расширение используется для привязки нескольких объектов SQL всех вместе к одному пакету, который может загружаться или удаляться из базы данных. Как только расширение загружено в базу данных, оно может функционировать как встроенная функция.

Триггер, зависящий от наличия расширения plpgsql:

ALTER TRIGGER some_trigger ON public.t DEPENDS ON EXTENSION plpgsql;



Включение/отключение триггера

Синтаксис:

```
ALTER TABLE table_name ENABLE/DISABLE TRIGGER trigger_name | ALL;
```

Пример: `ALTER TABLE public.t DISABLE TRIGGER some_trigger;`

Выключение триггера не удаляет его. Триггер по-прежнему существует, но находится в «спящем» состоянии. При выключении триггера он помечается красным крестиком в pgAdmin.

Включение: `ALTER TABLE public.t ENABLE TRIGGER some_trigger;`



Удаление триггера

Удаление триггера

Используйте DROP TRIGGER, чтобы удалить триггер PostgreSQL.

Синтаксис:

```
DROP TRIGGER [ IF EXISTS ] name ON table_name [ CASCADE | RESTRICT ]
```

IF EXISTS — указание на то, что не надо выдавать ошибку, если такого триггера нет.

CASCADE — автоматически удалять все объекты, которые зависят от триггера, объекты, которые зависят от этих объектов, и так далее.

RESTRICT — не удалять триггер, если от него зависят другие объекты. Это значение по умолчанию.



Доступ

Из pgAdmin:

Триггеры имеют отношение к таблицам и перечисляются в ветке каждой таблицы:

Servers->User Server->Databases->User Database->Schemas->

User Schema->Tables-> User Tables->Triggers

Из psql:

Получить список триггеров

\dS [имя_таблицы]



Задания (общие), блок 1

1. Создайте копии таблицы aircrafts: aircrafts_tmp и aircraft_log. Создайте триггер, который копирует в aircraft_log информацию о каждом новом самолете, добавляемом в aircrafts_tmp. Проверьте работу триггера, сохраните скриншоты результатов.
2. Создайте копии таблиц flights, aircrafts, seats. Напишите триггер, который выполняется перед удалением записи из таблицы aircrafts. Триггер проверяет наличие в таблицах flights и seats записей, относящихся к удаляемому самолету, и если такие записи есть, удаляет их. Проверьте работоспособность триггера, сохраните результаты.
3. Требуется обновлять общую сумму бронирования (total_amount в таблице bookings) при изменении данных о бронировании: изменении количества билетов, относящихся к бронированию, или стоимости билета (amount в ticket_flights) в бронировании. Создайте необходимые триггеры для автоматического обновления суммы бронирования при упомянутых изменениях. Проверьте работу триггеров, сохраните скриншоты результатов. Существует готовое решение в учебных материалах для вузов от Postgres Professional, которым можно воспользоваться. (См. в слайдах с описанием серверного программирования).



Задания, блок 2

Реализуйте триггеры для заданной базы данных, по вариантам.

Номер варианта вычисляется по формуле: $(\text{Номер_по_журналу} \% 20) + 1$, где $\%$ - остаток от деления.



Вариант 1 Склад

Необходимо хранить информацию о существующих складах (номер склада, адрес, телефон, фамилия руководителя склада), товарах (код, название, группа товара, фирма-производитель), а также о наличии товаров на конкретных складах с указанием количества товаров за последние три дня.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех склада из таблицы Склады, кроме склада с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - 1.2. Разработать триггер, который позволяет изменять значение количество товаров только на меньшее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
 - 2.1. Создать представление с информацией о Наличии товаров. Представление должно включать столбцы код товара, наименование товара, номер склада, Фамилию руководителя склада и количество товара на складе
 - 2.2. Разработать триггер, который позволяет вносить информацию о наличии товара через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем складе или товаре.

Схема базы данных

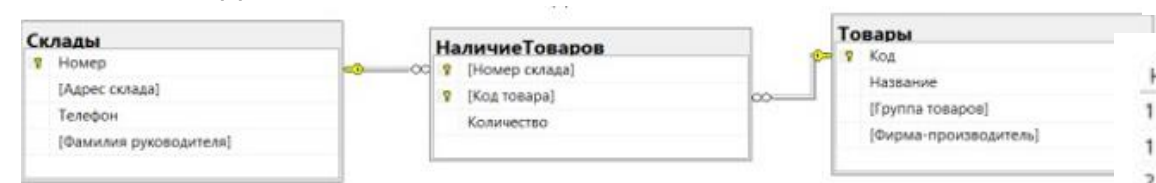


Таблица Склады

Номер	Адрес склада	Телефон	Фамилия руководителя
1	Первомайская, 1	111111	Иванов
2	Московское, 7	222222	Сидоров
3	Касимовское, 3	111222	Петров
4	Куйбышевское, 27	334455	Ковалев
5	Шабулина, 12	121212	Маматов
6	Яблочкова, 11	345678	Маматов
7	Циолковского, 17	778877	Сазонов
8	Павлова, 28	321321	NULL
9	Новоселов, 60	223344	Лоськов
10	Забайкальская, 14	445544	Родин

Таблица Товары

Код	Название	Группа товаров	Фирма-производитель
1	Колбаса	Продукты	Скопинский
2	Сыр	Продукты	Молкомбинат
3	Хлеб	Продукты	Хлебзавод 1
4	Телевизор	Техника	Sony
5	Стол	Мебель	IKEA
6	Автокресло	NULL	NULL
7	Лопата	Хозтовары	Мехзавод
8	Мочалка	Хозтовары	Авангард
9	Мыло	Хозтовары	Свобода
10	Кастрюля	Хозтовары	Технопласт

Таблица НаличиеТоваров

Номер склада	Код товара	Количество
1	3	5000
1	10	400
2	4	8000
2	7	1000
2	8	7000
3	3	500
3	8	9000
3	9	15000
4	1	2500
4	8	10000
5	2	1000
6	9	4500
6	3	2000
6	5	800
7	7	700
7	2	9000
8	1	3000
9	5	1000
9	8	5000
9	10	500



Схема базы данных

Вариант 2 Производство продуктов питания

Необходимо хранить информацию о фирмах-производителях продуктов (код фирмы, название фирмы, адрес, фамилия директора), продуктах (код, название, группа продуктов, вид упаковки), а также об объеме производства продуктов фирмами за последние три года.

1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.

1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо фирмах производителях, кроме фирмы с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.

1.2. Разработать триггер, который позволяет изменять значение объема производства только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.

2. Создание триггеров для представлений.

2.1. Создать представление с информацией о производство продуктов питания. Представление должно включать столбцы код фирмы, название фирмы, код продукта, название продукта и объем производства.

2.2. Разработать триггер, который позволяет вносить информацию о производстве продуктов питания через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем складе или товаре.

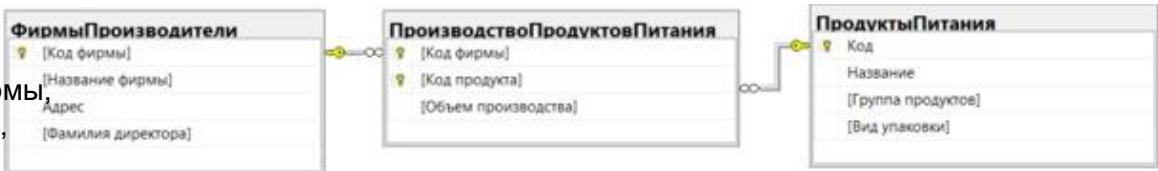


Таблица ФирмыПроизводители

Код фирмы	Название фирмы	Адрес	Фамилия директора
1	Шиловомясо	Шилово, ул. Рязанская, 118	Титкин
2	Вим Биль Дан	Москва, Дмитровское шоссе, 108	Пластинин
3	Рязаньхлеб	Рязань, ул. Военных автомобилистов, 3	Поляков
4	Агропищекомбинат Нива Рязани	Рязань, ул. Есенина, 9	Горкин
5	Кортлав	Рязань, ул. 14 линия, 2 стр. 1	Бренер
6	ОАО Лебедянский	Лебедянь, ул. Матросова, 7	Кобзев
7	ОАО Макфа	Челябинская область, г. Рошино	Юревич
8	Красный Октябрь	Москва, Берсеневская наб., 6	Даурской
9	ООО НЕП	С.-Петербург, ул. Тобольская, 3	Зуев
10	ПКП Русь	Петушки, ул. Клязьменская, д. 2	NULL

Таблица ПродуктыПитания

Код	Название	Группа продуктов	Вид упаковки
1	Сыр колбасный	Молочные	Целлофан
2	Молоко	Молочные	Картонная коробка
3	Хлеб бородинский	Хлебобулочные	Целлофан
4	Батон нарезной	Хлебобулочные	Целлофан
5	Батон окский	Хлебобулочные	NULL
6	Батон подмосковный	Хлебобулочные	Целлофан
7	Фруктовый сок	Питьевые	Картонная коробка
8	Макароны	Макаронные изделия	Целлофан
9	Капуста морская	Консервированные	Консервная банка
10	Шпроты рижские	Консервированные	Консервная банка

Таблица ПроизводствоПродуктовПитания

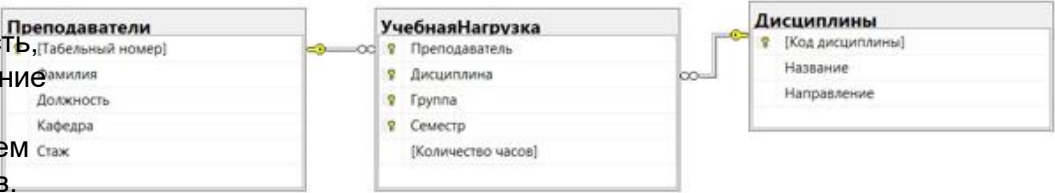
Код фирмы	Код продукта	Объем производства
1	2	100
2	2	250
2	7	300
3	3	500
3	4	750
3	6	180
3	5	260
4	1	1200
4	2	200
4	3	350
4	4	380
4	5	710
4	6	220
5	1	600
5	2	450
6	2	500
6	7	1400
9	3	1200
10	9	1600
10	10	120



Схема базы данных

Вариант 3 Учебная нагрузка

Необходимо хранить информацию о преподавателях (табельный номер, фамилия, должность, кафедра, стаж), дисциплинах (код, название, направление (гуманитарное, техническое и т.д.)), а также о распределении нагрузки по преподавателям с указанием номера группы студентов, семестра и количества часов.



- Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - Разработать AFTER-триггер, который позволяет удалять информацию преподавателей из таблицы Преподаватели, кроме преподавателя с табельным номером 11. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - Разработать триггер, который позволяет изменять значение стажа только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- Создание триггеров для представлений.
 - Создать представление с информацией об учебной нагрузке. Представление должно включать столбцы Табельный номер преподавателя, Фамилия, код дисциплины, название дисциплины, код группы, семестр и количество часов.
 - Разработать триггер, который позволяет вносить информацию об учебной нагрузке через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем преподавателе или дисциплине.

Таблица Преподаватели

Табельный номер	Фамилия	Должность	Кафедра	Стаж
11	Галкина	доцент	ЭВМ	8
12	Чичиков	доцент	ЭВМ	19
13	Иванов	доцент	ЭВМ	35
14	Орлов	NULL	ИОП	12
15	Туркин	доцент	САПР ВС	21
16	Чебышев	профессор	ИОП	15
17	Цаплина	NULL	ВМ	9
18	Павлушин	доцент	САПР ВС	2
19	Чечеткин	доцент	САПР ВС	5
20	Соловьев	ассистент	САПР ВС	NULL

Таблица УчебнаяНагрузка

Преподаватель	Дисциплина	Группа	Семестр	Количество часов
11	7	641	5	32
11	10	641	5	68
12	9	640	5	68
13	1	840	1	68
13	1	841	1	68
14	4	840	1	68
14	4	841	1	68
15	3	643	5	68
16	5	740	4	68
16	5	748	4	68
17	6	840	1	51
17	6	840	2	51
17	6	841	1	51
17	6	841	2	51
18	2	640	5	51
18	2	641	5	51
18	2	648	5	51
19	3	640	5	68
19	3	641	5	68
19	8	640	5	32

Таблица Дисциплины

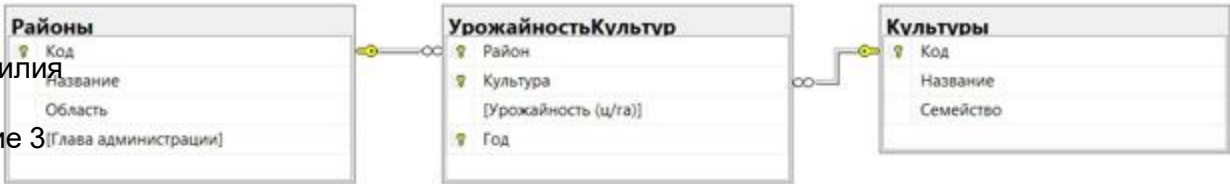
Код дисциплины	Название	Направление
1	Информатика	NULL
2	ЭиЭ	техническое
3	ИиКГ	NULL
4	История	NULL
5	Философия	гуманитарное
6	Математический анализ	математическое
7	Технологии программирования	техническое
8	ПУ ЭВМ	техническое
9	Операционные системы	техническое
10	Базы данных	техническое

Список заданий для выбора

Схема базы данных

Вариант 4 Растениеводство

Необходимо хранить информацию о районах различных областей (код, название района, название области, фамилия главы администрации района), культурах (код, название, семейство), а также об урожайности культур за последние 3 года.



1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.

1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех районах из таблицы Районы, кроме района с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.

1.2. Разработать триггер, который позволяет изменять значение урожайности только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.

2. Создание триггеров для представлений.

2.1. Создать представление с информацией о Урожайности культур. Представление должно включать столбцы код района, название района, код культуры, название культуры, год и урожайность.

2.2. Разработать триггер, который позволяет вносить информацию об урожайности культур через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем районе или культуре.

Таблица Районы

Код	Название	Область	Глава администрации
1	Сараевский	Рязанская	Толмачев
2	Сасовский	Рязанская	Рыбин
3	Пронский	Рязанская	Казаков
4	Спасский	Нижегородская	Евдокимов
5	Приаргунский	Читинская	Пичкуренко
6	Вельский	Архангельская	Колотилов
7	Ейский	Краснодарский край	NULL
8	Красноармейский	Краснодарский край	Тимофеев
9	Михайловский	Волгоградская	Семисотов
10	Кузоватовский	Ульяновская	Вильчик

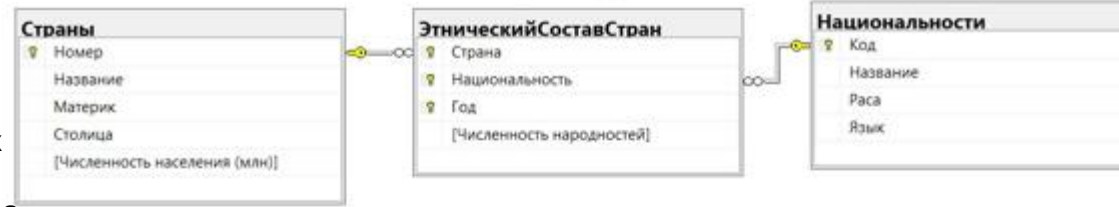
Таблица Культуры

Код	Название	Семейство
101	Пшеница	Злаки
102	Картофель	Пасленовые
103	Кукуруза	Злаки
104	Гречиха	Гречишные
105	Сахарная свекла	Маревые
106	Горох	Бобовые
107	Ячмень	Злаки
108	Рожь	Злаки
109	Рис	Злаки
110	Виноград	Виноградные

Таблица УрожайностьКультур

Район	Культура	Урожайность (ц/га)	Год
1	101	9	2018
1	107	16	2019
2	101	27	2018
2	102	141	2020
2	105	200	2019
2	108	17	2018
3	101	18	2019
3	102	130	2019
3	108	20	2018
4	101	15	2020
4	101	28	2018
6	102	173	2019
6	102	300	2018
7	103	61	2020
7	110	42	2020
8	109	40	2018
9	101	17	2018
9	103	67	2020
10	106	32	2019
10	108	30	2019

Схема базы данных



Вариант 5 Этнический состав стран

Необходимо хранить информацию о национальностях (код, название, раса, язык), странах (номер страны, название, материк, столица, численность населения), а также об этническом составе каждой отдельной страны (в тыс.) за последние три года.

Таблица Национальности

Код	Название	Раса	Язык
1	Русские	Европеоидная	Русский
2	Украинцы	Европеоидная	Украинский
3	Эстонцы	Европеоидная	Эстонский
4	Беларусы	Европеоидная	Белорусский
5	Хауса	Негроидная	Английский
6	Арабы	Европеоидная	Арабский
7	Немцы	Европеоидная	Немецкий
8	Монголы	Монголоидная	Монгольский
9	Китайцы	Монголоидная	Китайский
10	Евреи	Европеоидная	Иврит

Таблица ЭтническийСоставСтран

Страна	Национальность	Год	Численность народностей
1	1	2019	94000
1	6	2020	0,9
1	7	2019	1000
1	9	2019	900
1	9	2020	2000
1	10	2019	20000
2	1	2020	20000
2	3	2019	2
3	1	2019	300
3	9	2020	0,3
4	2	2020	2000
5	5	2020	60000
5	6	2019	10000
6	8	2018	1
7	6	2020	35000
7	10	2019	2000
8	4	2020	1
8	9	2018	1000
9	9	2019	1
10	7	2019	1

Таблица Страны

Номер	Название	Материк	Столица	Численность населения (мл)
1	Россия	Евразия	Москва	141
2	Украина	Евразия	Киев	43
3	Эстония	Евразия	NULL	NULL
4	Белоруссия	Евразия	Минск	9
5	Нигерия	Африка	Лагос	80
6	Алжир	Африка	Алжир	1
7	Египет	Африка	Каир	38
8	Канада	Северная Америка	Оттава	23
9	Монголия	Евразия	Улан-Батор	1,5
10	Китай	Евразия	Пекин	1000

1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.

1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех странах из таблицы Страны, кроме страны с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.

1.2. Разработать триггер, который позволяет изменять значение численности населения только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.

2. Создание триггеров для представлений.

2.1. Создать представление с информацией о этническом составе стран. Представление должно включать столбцы номер страны, название страны, код национальности, название национальности, год и численность населения

2.2. Разработать триггер, который позволяет вносить информацию об этническом составе стран через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующей национальности или стране.



Схема базы данных

Вариант 6 Курсы иностранных языков

Необходимо хранить информацию об учащихся (табельный номер, фамилия, год рождения, контактный телефон), предлагаемых языках (код, название, языковая группа, фамилия руководителя направления), а также о посещении учащимися занятий по конкретным языкам с указанием даты начала занятий, длительности и стоимости курсов.

- Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - Разработать AFTER-триггер, который позволяет удалять информацию обо всех языках из таблицы Языки, кроме язык с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - Разработать триггер, который позволяет изменять значение длительности только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- Создание триггеров для представлений.
 - Создать представление с информацией о Посещение занятий. Представление должно включать столбцы табельный номер, фамилия, код, название, начало занятий, длительность и стоимость.
 - Разработать триггер, который позволяет вносить информацию о посещении занятий через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем языке или учащемся.

Схема базы данных

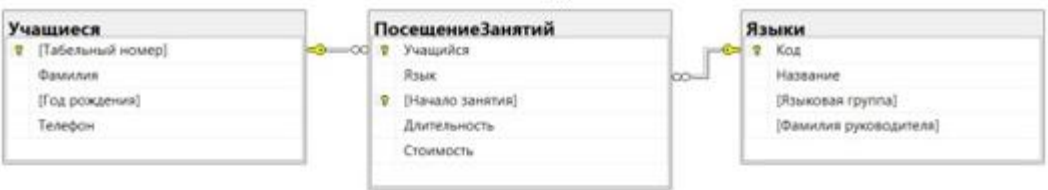


Таблица Языки

Код	Название	Языковая группа	Фамилия руководителя
1	Английский	Западногерманская	Рюмин
2	Польский	Славянская	Муравьев
3	Испанский	Романская	Балабанов
4	Русский	Славянская	Гасперт
5	Итальянский	Романская	Муравьев
6	Французский	Романская	Моле
7	Немецкий	Западногерманская	Исаев
8	Латинский	Италийская	Бродский
9	Шведский	Скандинавская	Бергман
10	Чешский	Славянская	Чудов

Таблица Учащиеся

Табельный номер	Фамилия	Год рождения	Телефон
1	Иванов	1980	112211
2	Гришин	1975	554637
3	Морозов	1985	NULL
4	Троцкий	1992	346712
5	Медведев	1983	543321
6	Гусев	1988	543622
7	Кудрин	1993	234214
8	Разин	1979	554325
9	Пугачев	1990	367162
10	Болотников	1987	123435

Таблица ПосещениеЗанятий

Учащийся	Язык	Начало занятия	Длительность	Стоимость
1	7	2020-02-25 15:00:00.000	2	200,0000
1	7	2020-04-02 12:00:00.000	1	100,0000
2	2	2020-02-05 11:00:00.000	2	300,0000
2	2	2020-05-12 09:00:00.000	3	450,0000
5	6	2020-05-01 10:00:00.000	2	450,0000
4	9	2020-02-05 14:00:00.000	5	5000,0000
4	9	2020-05-30 12:00:00.000	5	5000,0000
5	8	2020-04-05 14:00:00.000	4	4000,0000
5	8	2020-05-06 09:00:00.000	2	2000,0000
6	10	2020-01-28 20:00:00.000	2	NULL
7	4	2020-02-20 15:00:00.000	3	250,0000
7	3	2020-05-10 13:00:00.000	4	2250,0000
7	7	2020-05-14 15:00:00.000	3	500,0000
7	4	2020-10-20 12:00:00.000	2	NULL
8	4	2020-02-02 17:00:00.000	1	100,0000
8	4	2020-08-05 14:00:00.000	3	2000,0000
9	5	2020-08-22 14:00:00.000	1	300,0000
10	4	2020-02-01 11:00:00.000	2	500,0000
10	5	2020-02-01 15:00:00.000	4	2000,0000
10	3	2020-07-03 18:00:00.000	3	2000,0000

Вариант 7 Услуги парикмахера

Необходимо хранить информацию о предоставляемых в салоне видах услуг (код, название, цена), клиентах (фамилия, адрес, контактный телефон, год рождения), а также о предварительной записи клиентов на услуги с указанием даты и времени посещения.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех услугах из таблицы Услуги, кроме услуги с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - 1.2. Разработать триггер, который позволяет изменять значение цены только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
 - 2. Создание триггеров для представлений.
 - 2.1. Создать представление с информацией о Записи клиентов на услуги. Представление должно включать столбцы номер карты, фамилия клиента, код услуги, название, дата и время.
 - 2.2. Разработать триггер, который позволяет вносить информацию о записи клиентов на услуги через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление.
- Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем клиенте или услуге.

Схема базы данных

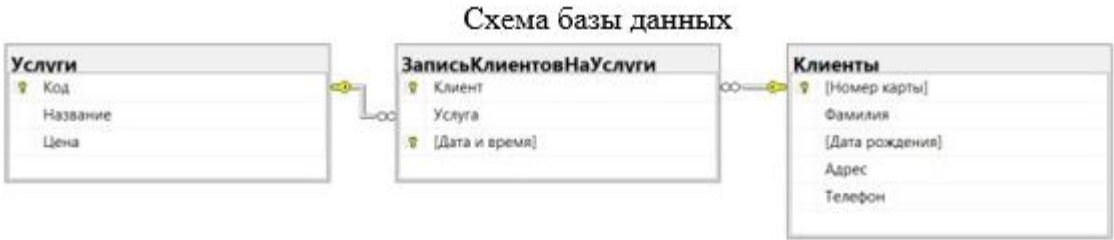


Таблица Услуги

Код	Название	Цена
1	Стрижка	150,0000
2	Мелирование	750,0000
3	Покраска	200,0000
4	Мытье головы	100,0000
5	Сушка	100,0000
6	Лечение волос	1000,0000
7	Плетение косичек	500,0000
8	Торжественная прическа	800,0000
9	Укладка	200,0000
10	Бритье	50,0000

Таблица Клиенты

Номер карты	Фамилия	Дата рождения	Адрес	Телефон
1	Иванова	1975-10-15	Тимокова, 5	325531
2	Петров	1970-08-05	Зубкова, 6	316940
3	Сидорова	1980-11-10	Гоголя, 26	418122
4	Гуляев	1977-05-11	Лермонтова, 13	775020
5	Жукова	1989-02-08	NULL	333911
6	Васечкин	1990-10-06	Циолковского, 10	NULL
7	Соколова	1985-12-20	Гагарина, 53	779750
8	Орлов	1988-06-22	Дзержинского, 18	589551
9	Бугреева	1988-07-30	Спортивная, 15	NULL
10	Уткин	1979-01-25	Есенина, 20	321520

Таблица ЗаписьКлиентовНаУслуги

Клиент	Услуга	Дата и время
2	4	2020-03-01 12:10:00.000
2	5	2020-03-01 15:00:00.000
3	1	2020-03-02 10:20:00.000
3	9	2020-04-03 15:20:00.000
4	2	2020-04-03 17:00:00.000
5	1	2020-06-05 10:00:00.000
5	3	2020-06-05 12:00:00.000
5	4	2020-09-10 17:00:00.000
5	4	2020-10-10 12:00:00.000
6	1	2020-11-12 12:00:00.000
6	10	2020-11-15 10:00:00.000
7	6	2020-11-16 12:00:00.000
9	7	2020-12-19 10:00:00.000
8	4	2020-12-20 10:00:00.000
8	3	2020-12-20 17:00:00.000
8	5	2020-12-29 10:00:00.000
3	8	2020-12-31 10:00:00.000
5	8	2020-12-31 12:00:00.000
7	8	2020-12-31 15:00:00.000
9	8	2020-12-31 17:00:00.000

Вариант 8 Ателье по пошиву и ремонту одежды

Необходимо хранить информацию о существующем ателье (номер ателье, название, адрес, телефон), видах предоставляемых услуг (код, название, длительность выполнения), а также о стоимости таких услуг в конкретном ателье за последние 3 месяца.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех ателье из таблицы Ателье, кроме ателье с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - 1.2. Разработать триггер, который позволяет изменять значение длительности выполнения только на меньшее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
 - 2.1. Создать представление с информацией о стоимости. Представление должно включать столбцы номер ателье, название ателье, код услуги, название услуги, стоимость
 - 2.2. Разработать триггер, который позволяет вносить информацию о стоимости через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем ателье или услуге.



Таблица Ателье

Номер	Название	Адрес	Телефон
1	Золотое Руно	Новая, 28/32	250085
2	Золушка	Разина, 2А	445791
3	Имидж	Карла Маркса,11	289622
4	Канесса	Энгельса, 35	445044
5	Карина	Дзержинского, 40	NULL
6	Кутюрье	Халтурина, 1Б	324088
7	Макошь	Фрунзе, 15	456077
8	Миранда-А	Крупской, 27	557890
9	Мода	ЛенКом, 13	296057
10	Образ	Урицкого, 25	411840

Таблица Услуги

Код	Название	Длительность выполнения
1	Укорачивание	10
2	Подгонка	22
3	Корректировка	29
4	Перекрой	30
5	Изменение фасона	21
6	Замена подкладки	15
7	Замена фурнитуры	8
8	Вышивка	NULL
9	Пошив	35
10	Штопка	17

Таблица Стоимость

Ателье	Услуга	Стоимость
1	1	400,0000
1	2	850,0000
1	3	700,0000
1	4	1000,0000
2	1	600,0000
2	10	200,0000
4	6	500,0000
4	7	60,0000
5	9	3800,0000
5	10	200,0000
6	2	780,0000
7	4	670,0000
7	5	900,0000
7	6	540,0000
8	7	100,0000
8	9	4200,0000
9	5	1100,0000
9	9	7800,0000
10	1	500,0000
10	9	400,0000

Вариант 9 Красная книга растений

Необходимо хранить информацию о редких растениях (название, семейство, раздел), странах (название, материк, столица), а также о произрастании растений в отдельных странах с указанием приблизительного количества корней за последние три года.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех растениях из таблицы Растения, кроме растения с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - 1.2. Разработать триггер, который позволяет изменять значение количества корней только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
 - 2.1. Создать представление с информацией о растения в стране. Представление должно включать столбцы код растения, название, код страны, название страны, количество корней.
 - 2.2. Разработать триггер, который позволяет вносить информацию о растениях в стране через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем растении или стране.

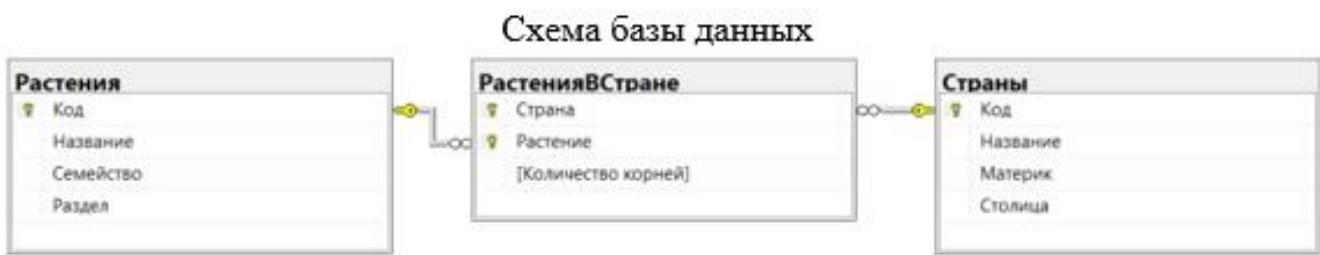


Таблица Растения

Код	Название	Семейство	Раздел
1	Мутинус собачий	Веселковые	Грибы
2	Рогатик пестиковый	Клавариевые	Грибы
3	Пиррозия язычная	Многоножковые	Папоротниковидные
4	Лепторумора Микеля	Аспидиевые	Папоротниковидные
5	Марсила щетинистая	Марсилевые	Папоротниковидные
6	Стереокаулон обнаженный	Стереокаулиевые	NULL
7	Асахиния Шоландера	Пармелиевые	Лишайники
8	Кладония вулканная	Кладониевые	Лишайники
9	Корникулярия степная	Уснеевые	Лишайники
10	Телосхистес Желтоватый	Телосхистовые	Лишайники

Таблица Страны

Код	Название	Материк	Столица
1	Россия	Евразия	Москва
2	Чехия	Евразия	Прага
3	Германия	Евразия	Берлин
4	Польша	Евразия	Варшава
5	Франция	Евразия	Париж
6	Австралия	Австралия	NULL
7	Латвия	Евразия	Рига
8	Канада	Северная Америка	Оттава
9	Мексика	Северная Америка	Мехико
10	Нигерия	Африка	Абуджа

Таблица РастенияВСтране

Страна	Растение	Количество корней
1	3	300
3	1	10
4	2	500
5	2	230
5	5	100
9	9	56
1	6	400
6	5	100
1	2	560
5	1	20
7	7	500
8	9	140
9	10	56
2	5	122
3	5	166
3	4	122
4	9	200
5	4	20
6	1	145
7	2	546

Вариант 10 Животноводство

Необходимо хранить информацию о совхозах области (код, название совхоза, название района, фамилия председателя), видах скота (код, название вида, порода), а также о поголовье скота в различных совхозах за последние три года.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
- 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех совхозах из таблицы совхозы, кроме совхоза с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
- 1.2. Разработать триггер, который позволяет изменять значение поголовья только на меньшее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
- 2.1. Создать представление с информацией о поголовье. Представление должно включать столбцы код скота, название вида, код совхоза, название совхоза, дата сбора статистики и поголовье.
- 2.2. Разработать триггер, который позволяет вносить информацию о поголовье через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем скоте или совхозе.

Схема базы данных

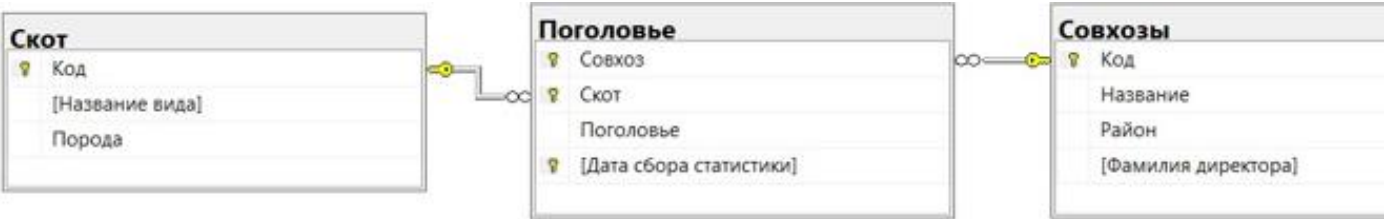


Таблица Совхозы

Код	Название	Район	Фамилия директора
1	Знамя	Касимовский	Орехов
2	Заря	Рыбновский	Слюков
3	Искра	Шацкий	Петров
4	Воля	Рязанский	Попов
5	Урожай	Рязанский	Баев
6	Авангард	Скопинский	Иванов
7	Победа	Пронский	Зотов
8	Успех	Спасский	Рынин
9	Сила	NULL	Пылев
10	Звезда	Сараевский	Зайцев

Таблица Скот

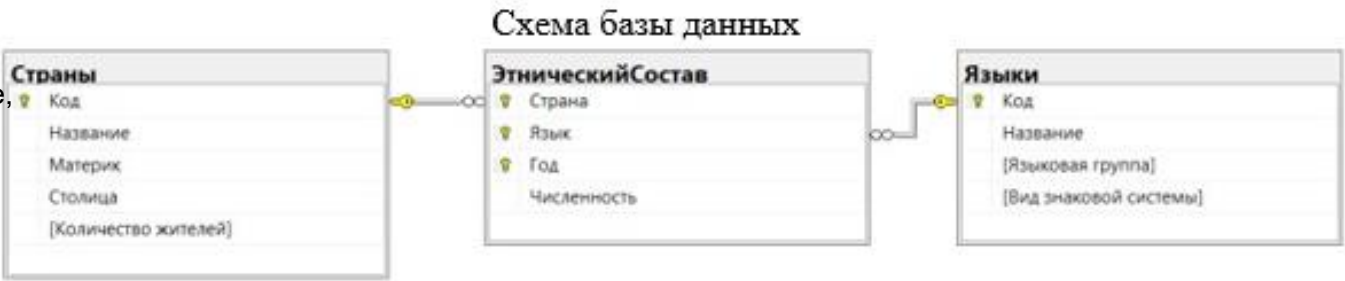
Код	Название вида	Порода
1	Бык	Черный
2	Бык	Красный
3	Лошадь	Киргизская
4	Лошадь	Калмыцкая
5	Свинья	Белая
6	Овца	Грубошерстная
7	Курица	Минорка
8	Утка	Пекинская
9	Свинья	Эстонская
10	Овца	Тонкорунная

Таблица Поголовье

Совхоз	Скот	Поголовье	Дата сбора статистики
1	2	100	2020-01-01
1	5	110	2020-01-07
2	4	95	2020-01-01
2	6	50	2020-02-03
3	1	55	2020-03-01
3	6	220	2020-05-04
4	3	200	2020-01-12
4	8	80	2020-07-23
5	5	75	2020-04-14
5	10	45	2020-09-17
6	2	130	2020-01-01
6	4	111	2020-08-19
7	6	120	2020-01-01
7	7	150	2020-11-21
7	9	91	2020-01-31
8	7	80	2020-01-12
10	1	150	2020-09-07
10	4	90	2020-05-11
10	8	70	2020-06-18
10	10	60	2020-07-22

Вариант 11 Языки народов мира

Необходимо хранить информацию о языках (код, название, языковая группа, вид знаковой системы (кириллица, латиница, иероглифы и т.п.)), странах (код страны, название, материк, столица, количество жителей), а также об этническом составе каждой отдельной страны за последние 3 года с указанием численности населения, говорящего на каждом из языков.



1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.

1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех странах из таблицы страны, кроме страны с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.

1.2. Разработать триггер, который позволяет изменять значение количества жителей только на меньшее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.

2. Создание триггеров для представлений.

2.1. Создать представление с информацией об этническом составе. Представление должно включать столбцы код страны, название, код языка, название, год и численность.

2.2. Разработать триггер, который позволяет вносить информацию об этническом составе через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующей стране или языке.

Таблица Страны

Код	Название	Материк	Столица	Количество жителей
1	Россия	Евразия	Москва	143782000
2	Франция	Евразия	Париж	60424000
3	США	Северная Америка	Вашингтон	293027000
4	Германия	Евразия	Берлин	82424000
5	Болгария	Евразия	София	7640000
6	Швеция	Евразия	Стокгольм	9100000
7	Япония	NULL	NULL	127333000
8	Китай	Евразия	Пекин	1300000000
9	Украина	Евразия	NULL	47732000
10	Канада	Северная Америка	Оттава	32507000

Таблица Языки

Код	Название	Языковая группа	Вид знаковой системы
1	русский	славянская	кириллица
2	украинский	славянская	кириллица
3	белорусский	славянская	кириллица
4	французский	романская	латиница
5	американский	германская	латиница
6	немецкий	германская	NULL
7	болгарский	славянская	NULL
8	шведский	германская	латиница
9	японский	японская	иероглифы
10	китайский	сино-тибетская	иероглифы

Таблица ЭтническийСостав

Страна	Язык	Год	Численность
1	1	2020	143000000
1	2	2019	500000
1	2	2020	600000
1	10	2018	30000
1	10	2019	34000
1	10	2020	40000
2	4	2019	59000000
2	4	2020	60000000
3	5	2018	290000000
3	5	2020	292000000
4	1	2019	20000
4	6	2019	82000000
5	7	2020	7600000
6	8	2019	9000000
6	8	2020	9100000
7	9	2020	127000000
8	10	2018	1299000000
8	10	2020	1300000000
10	4	2018	30000000
10	4	2020	32000000

Вариант 12 Сеть компьютерных магазинов города

Необходимо хранить информацию о существующих магазинах (номер магазина, название, адрес, телефон), комплектующих (модель, название вида, фирма-производитель, цена), а также о наличии комплектующих в конкретных магазинах с указанием их количества за последние три года.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех магазинах а из таблицы Магазины, кроме магазина с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - 1.2. Разработать триггер, который позволяет изменять значение цены товаров только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
 - 2. Создание триггеров для представлений.
 - 2.1. Создать представление с информацией о наличие комплектующих. Представление должно включать столбцы модель, название, номер, название магазина, год, количество
 - 2.2. Разработать триггер, который позволяет вносить информацию о наличии комплектующих через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление.
- Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем магазине или комплектующем.

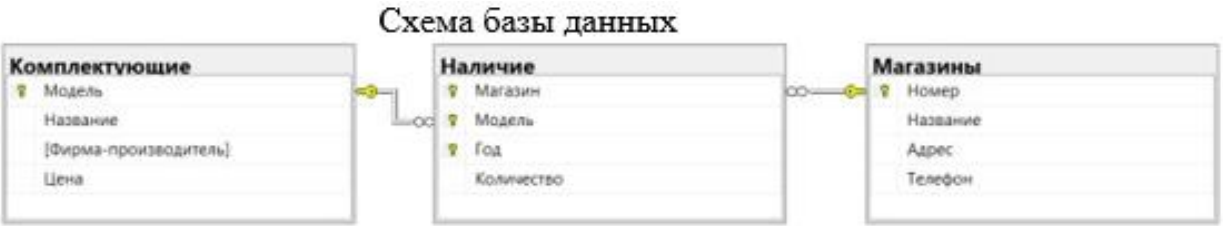


Таблица Комплектующие

Модель	Название	Фирма-производитель	Цена
22	Очки для компьютера	MATSUDA	700,0000
27	Компьютерная мышь	SAMSUNG	500,0000
113	Компьютерная мышь	BENQ	400,0000
222	Клавиатура	NULL	530,0000
276	Привод DVD+RW	Asus	1000,0000
336	Ноутбук	BENQ	36000,0000
377	Монитор	Acer	6000,0000
576	Колонки	DAEWOO	3500,0000
900	Чистящие салфетки	NULL	250,0000
3000	Коврик для мыши	NULL	100,0000
5100	Ноутбук	HP	27000,0000

Таблица Наличие

Магазин	Модель	Год	Количество
1	336	2019	50
1	336	2020	70
2	576	2018	60
3	377	2019	20
3	576	2018	30
3	5100	2019	40
3	5100	2020	30
4	222	2018	63
5	3000	2018	300
5	3000	2019	350
5	3000	2020	250
6	113	2019	300
6	576	2019	90
8	377	2018	40
8	900	2018	300
9	22	2019	200
9	22	2020	250
10	222	2018	90
10	222	2019	80
10	222	2020	100

Таблица Магазины

Номер	Название	Адрес	Телефон
1	Компьютерный мир	Советская, 36	336772
2	Техносила	Грибоедова, 32	554411
3	НИКС	Полегаева, 104	123456
4	Эльдорадо	Солнечная, 2	5561789
5	Техномир	Гагарина, 4	NULL
6	Формоза	Циолковского, 75	757522
7	Альт	Свободы, 99	276349
8	От А до Я	Новоселов, 22	NULL
9	НИТИ	Октябрьская, 83	NULL
10	Все для компьютера	Интернациональная, 44	192837

Вариант 13 Туристические фирмы города

Необходимо хранить информацию о существующих в городе фирмах (код, название, адрес, фамилия директора), постоянных клиентах (табельный номер, фамилия, адрес, телефон), а также о турах, заказанных клиентами с указанием страны, даты отъезда и стоимости тура.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
- 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех фирмах из таблицы Фирмы, кроме фирмы с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
- 1.2. Разработать триггер, который позволяет изменять значение стоимости тура только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
- 2.1. Создать представление с информацией о туристических турах. Представление должно включать столбцы табельный номер клиента, фамилия, код фирмы, название, дата отъезда, страна, стоимость тура.
- 2.2. Разработать триггер, который позволяет вносить информацию о туристическом туре через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующей фирме или клиенте.

Схема базы данных

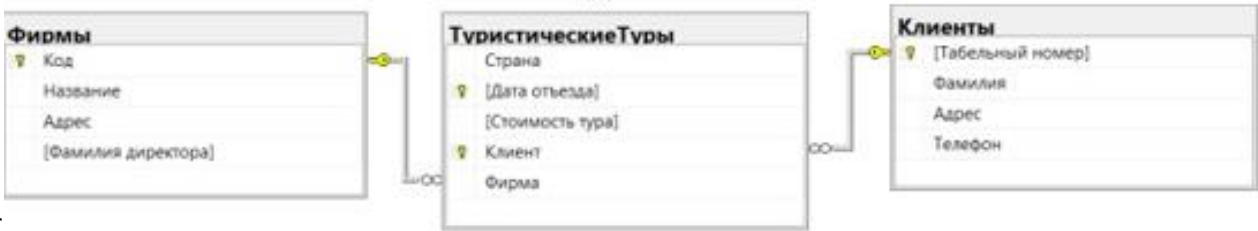


Таблица Фирмы

Код	Название	Адрес	Фамилия директора
1	Алые паруса	Вокзальная, 11	Шахов
2	Арго	Есенина, 72	Маркова
3	БризТур	Ленина, 24	NULL
4	Визази	Ленина, 10	Хрипин
5	Гулливер	NULL	Горскина
6	Компас	Первомайский пр-т, 15	Воронин
7	Ликтур	Колхозный пр-т, 15	Звягина
8	Колумб	Первомайский пр-т, 82	Бурмистров
9	Магеллан	Вокзальная, 34	Колткова
10	Спутник	Садовая, 22	Осипова

Таблица Клиенты

Табельный номер	Фамилия	Адрес	Телефон
1	Бараненков	Сенная, 8	256397
2	Кузнецова	Маяковского, 9	765220
3	Макаренко	Соборная, 15	281099
4	Гусев	Красноярская, 3	902190
5	Крюкова	Гагарина, 77	678243
6	Митасов	Почтовая, 58	152976
7	Азарьева	Кольцова, 1	148934
8	Данилова	Горького, 17	910543
9	Воронцов	Введенская, 110	239844
10	Афанасьева	Татарская, 15	234588

Таблица ТуристическиеТуры

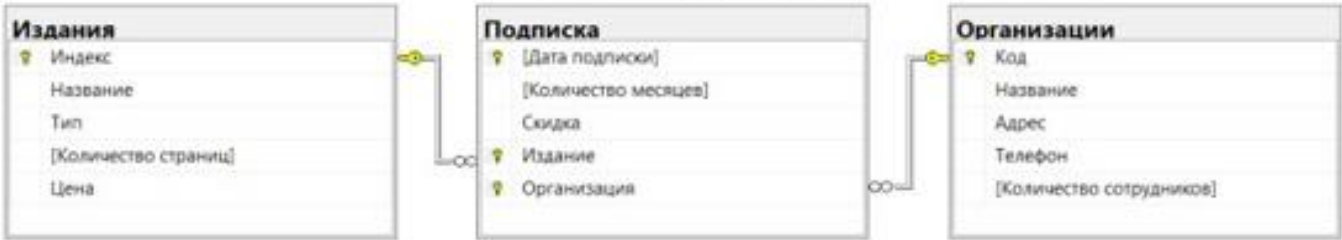
Страна	Дата отъезда	Стоимость тура	Клиент	Фирма
Абхазия	2020-07-10	2000.0000	7	8
Австрия	2020-08-29	30000.0000	8	3
ОАЭ	2020-01-17	20000.0000	10	2
Болгария	2020-01-01	12000.0000	1	5
Великобритания	2020-11-07	18000.0000	4	9
Германия	2020-03-13	15000.0000	8	8
Греция	2020-11-19	25000.0000	1	1
Египет	2020-04-27	35000.0000	4	1
Индонезия	2020-10-15	39000.0000	10	4
Испания	2020-02-02	57000.0000	1	2
Италия	2020-09-13	55000.0000	6	8
Кипр	2020-02-14	40000.0000	9	5
Россия	2020-04-27	10000.0000	2	10
Тунис	2020-09-18	42000.0000	9	3
Турция	2020-12-22	30000.0000	2	9
Финляндия	2020-04-27	40000.0000	7	6
Франция	2020-05-30	27000.0000	5	6
Хорватия	2020-10-08	13000.0000	5	4
Чехия	2020-03-25	14000.0000	2	1
Швейцария	2020-06-06	50000.0000	6	1



Вариант 14 Подписка на периодические издания организациями города

Необходимо хранить информацию о существующих организациях города (код, название, адрес, телефон, количество сотрудников), о распространяемых изданиях (индекс, название, тип [газета, журнал и т.п.], количество страниц, цена), а также о подписке организаций на издания с указанием даты начала подписки, количества месяцев и скидки на стоимость подписки в процентах.

Схема базы данных



1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.

1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех организациях из таблицы Организации, кроме организации с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.

1.2. Разработать триггер, который позволяет изменять значение размера скидки только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.

2. Создание триггеров для представлений.

2.1. Создать представление с информацией о подписке. Представление должно включать столбцы индекс, название издания, код, название организации, дата подписки, количество месяцев, скидка.

2.2. Разработать триггер, который позволяет вносить информацию о подписке через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление.

Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем издании или организации.

Таблица Организации

Код	Название	Адрес	Телефон	Количество сотрудников
1	ООО Мо6Про	Декабристов, 12	745504	100
2	ООО Опекс	Ленинградский пр-т, 62	771619	50
3	ООО Ростидромет	Кутузовский пр-т, 26	NULL	30
4	ООО Балттекстиль	Мичуринский пр-т, 3	925116	150
5	ООО СтройДом	Ленина, 3	283455	40
6	ЗАО Виаско	Первомайский пр-т, 7	695836	15
7	ООО Интертехно	Гагарина, 5	NULL	10
8	ООО Авиценна	Почтовая, 9	678433	7
9	ООО Артурпроект	Полевая, 2	NULL	8
10	ОАО Биопрепарат	Первомайский пр-т, 14	678723	9

Таблица Издания

Индекс	Название	Тип	Количество страниц	Цена
1	Панорама	газета	30	12,0000
2	Телесемь	газета	40	15,0000
3	I Love You	NULL	60	30,0000
4	Лиза	журнал	70	35,0000
5	Из рук в руки	газета	50	7,0000
6	Ярмарка	NULL	100	10,0000
7	Maxim	журнал	150	90,0000
8	Мир новостей	газета	70	15,0000
9	Cosmo	журнал	100	100,0000
10	Yoi	журнал	80	60,0000

Таблица Подписка

Дата подписки	Количество месяцев	Скидка	Издание	Организация
2020-01-06	5	0,02	1	3
2020-01-06	3	0,01	9	5
2020-01-08	7	0,12	1	7
2020-02-21	3	0	2	5
2020-03-15	3	0,02	2	8
2020-03-29	7	0	3	6
2020-04-03	4	0,5	3	10
2020-04-12	1	0,01	4	1
2020-04-14	8	0,02	4	10
2020-05-10	12	0,13	4	6
2020-05-17	10	0,11	3	7
2020-06-12	1	0,01	6	2
2020-06-19	8	0	9	3
2020-07-24	6	0	7	3
2020-08-30	3	0	6	8
2020-09-07	2	0,02	6	1
2020-09-16	5	0,01	1	9
2020-10-03	3	0	7	2
2020-11-28	2	0	10	7
2020-12-01	6	0,01	10	6

Вариант 15 Расписание занятий в вузе

Необходимо хранить информацию о дисциплинах (код, название, количество часов), о преподавателях (табельный номер, ФИО, должность, возраст, телефон), а также о проведении занятий с указанием дня недели, времени, группы и аудитории.

- Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - Разработать AFTER-триггер, который позволяет удалять информацию обо преподавателях из таблицы Преподаватели, кроме преподавателя с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - Разработать триггер, который позволяет изменять значение возраста только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- Создание триггеров для представлений.
 - Создать представление с информацией о Занятиях. Представление должно включать столбцы табельный номер, ФИО, код, название, день недели, время, группа и аудитория
 - Разработать триггер, который позволяет вносить информацию о занятии через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем преподавателе или дисциплине.

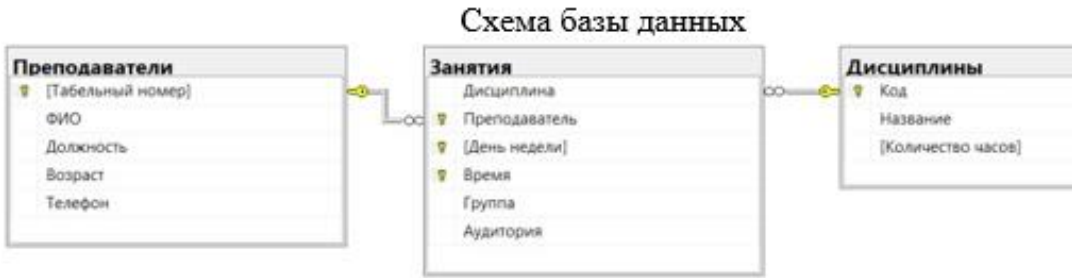


Таблица Дисциплины

Код	Название	Количество часов
1	Базы данных	64
2	Экономика	32
3	Математический анализ	51
4	Философия	NULL
5	Программирование	64
6	Физика	32
7	Математическая логика	51
8	Организация ЭВМ и систем	64
9	Информатика	64
10	Основы ИТ	32

Таблица Занятия

Дисциплина	Преподаватель	День недели	Время	Группа	Аудитория
1	1	Вторник	15:20:00	643	210
1	1	Понедельник	09:55:00	640	210
5	1	Среда	11:40:00	641	210
9	1	Четверг	13:35:00	648	210
2	2	Понедельник	09:55:00	740	268
2	2	Пятница	08:10:00	721	312
3	3	Пятница	11:40:00	741	466
3	3	Четверг	13:35:00	748	310
4	4	Четверг	13:35:00	745	311
5	5	Вторник	11:40:00	543	206
5	5	Понедельник	09:55:00	541	206
6	6	Вторник	11:40:00	746	358
7	7	Пятница	11:40:00	748	333
7	7	Четверг	09:55:00	740	333
5	8	Пятница	08:10:00	740	208
8	8	Среда	09:55:00	741	210
8	9	Вторник	11:40:00	840	208
9	9	Среда	09:55:00	841	208
8	9	Вторник	09:55:00	740	206
5	9	Среда	15:20:00	743	206

Таблица Преподаватели

Табельный номер	ФИО	Должность	Возраст	Телефон
1	Галкина Наталья Николаевна	Доцент	26	345519
2	Смирнов Максим Анатольевич	Доцент	46	255563
3	Майоров Валентин Васильевич	Профессор	30	211577
4	Шавелев Анатолий Анатольевич	Доцент	41	554812
5	Пушкин Александр Николаевич	Доцент	50	985601
6	Орлов Александр Павлович	Доцент	51	475056
7	Павлов Александр Викторович	Доцент	31	448003
8	Есенина Светлана Ивановна	Доцент	32	389908
9	Устинов Дмитрий Игоревич	Старший преподаватель	30	542132
10	Белкин Андрей Витальевич	Доцент	40	219965



Вариант 16 Справочная служба аптек

Необходимо хранить информацию о существующих аптеках города (номер, название, телефон, адрес, ФИО заведующего), о поставляемых в аптеки лекарствах (код, название, масса, группа (антибиотики, антисептики, сульфаниламиды, анальгетики и т.п.)), а также о наличии лекарств в аптеках с указанием количества упаковок и цены за последние 3 дня.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех аптеках из таблицы Аптеки, кроме аптеки с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - 1.2. Разработать триггер, который позволяет изменять значение цены только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
 - 2.1. Создать представление с информацией о Наличии. Представление должно включать столбцы код лекарства, название, номер аптеки, название аптеки, количество, цена.
 - 2.2. Разработать триггер, который позволяет вносить информацию о наличии через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующей аптеки или лекарства.

Схема базы данных

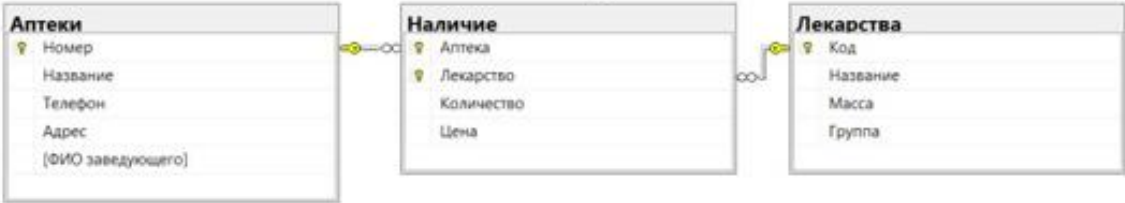


Таблица Аптеки

Номер	Название	Телефон	Адрес	ФИО заведующего
1	Спрей	354678	Первомайский пр.	Иванова М.И.
2	Здоровье	332211	Энгельса, 6	Семущева Е.Е.
3	Парацельс	666666	Ленина, 35	Рычкова И.Г.
4	Солнышко	123456	Белякова, 8	Герман Е.В.
5	Центральная	654321	Октябрьская, 13	Пилюлькин Е.Ц.
6	Капелька	134223	Забайкальская, 12	Дынькин К.Г.
7	Айболит	124355	Гагарина, 23	Кац Ц.Б.
8	Неболей	245376	Горького, 46	Малахов Г.П.
9	Намибия	243462	Есенина, 23	Тобеныхина Н.И.
10	Добродей	125433	Островского, 11	Черных Ю.К.

Таблица Лекарства

Код	Название	Масса	Группа
1	Диоксидин	0,8	Антисептики
2	Парацетамол	0,6	Анальгетики
3	Тетрациклин	0,5	Антибиотики
4	Амоксиклав	1	Антибиотики
5	Пенициллин	0,5	Антибиотики
6	Фенозепам	1	Анксиолитики
7	Этазол	1	Сульфаниламиды
8	Аспирин	0,5	NULL
9	Уросульфам	0,8	Сульфаниламиды
10	Ципролет	1	Антибиотики

Таблица Наличие

Аптека	Лекарство	Количество	Цена
1	1	50	200,0000
1	2	120	100,0000
2	8	100	140,0000
2	5	50	72,0000
3	1	70	220,0000
3	2	15	95,0000
3	7	30	450,0000
4	8	20	30,0000
4	6	40	90,0000
6	4	15	20,0000
6	5	55	65,0000
7	9	100	190,0000
7	4	30	25,0000
8	9	60	201,0000
8	10	20	80,0000
9	7	1	500,0000
9	8	15	70,0000
9	9	30	197,0000
10	5	12	71,0000
10	10	27	103,0000

Вариант 17 Сеть магазинов по продаже бытовой техники

Необходимо хранить информацию о существующих магазинах (номер, адрес, телефон, Ф.И.О. директора, количество сотрудников), о продаваемой технике (код, название, марка, масса, цена), а также о наличии техники в магазинах с указанием количества за последние три дня.

- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
 - 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо магазинах из таблицы магазины, кроме магазина с номером 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
 - 1.2. Разработать триггер, который позволяет изменять значение цены товаров только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
 - 2.1. Создать представление с информацией о наличии техники в магазине (Наличие_техники_в). Представление должно включать столбцы код техники, название техники, номер магазина, ФИО директора, количество
 - 2.2. Разработать триггер, который позволяет вносить информацию о наличии техники через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем магазине или технике.

Схема базы данных

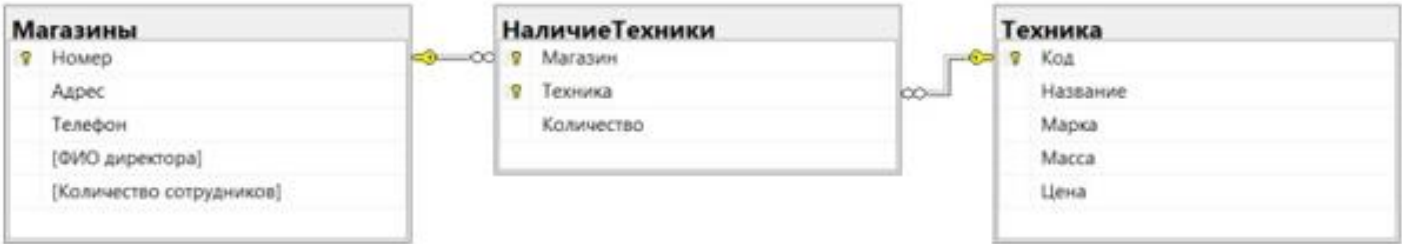


Таблица Магазины

Номер	Адрес	Телефон	ФИО директора	Количество сотрудников
1	Гагарина, 34	112233	Лапин М.Н.	50
2	Народный бульвар, 12	223311	Морозов К.Л.	30
3	Ленина, 18	443322	Калинина Ю.Д.	45
4	Зубковой, 56	772233	Шац П.А.	60
5	Циолковского, 43а	225533	Кокеткина И.Н.	35
6	Московское шоссе, 76	225544	Барсов Р.К.	75
7	Грибоедова, 1	555555	Васильев М.Т.	45
8	Интернациональная, 12	771122	Гагарин Ю.Д.	50
9	Колхозная, 33	553344	Кратер Р.Л.	30
10	Западная, 12	445566	Савельев П.Н.	45

Таблица Техника

Код	Название	Марка	Масса	Цена
1	Холодильник	Samsung	50	12500,0000
2	Пылесос	Bosch	5	6200,0000
3	Мультиварка	Samsung	10	17800,0000
4	Робот-пылесос	Samsung	5	25000,0000
5	Мясорубка	Bosch	3	2300,0000
6	Телевизор	LG	30	100000,0000
7	Музыкальный центр	Pioneer	NULL	8000,0000
8	Миксер	Braun	2	1500,0000
9	Стиральная машина	Ariston	3	12500,0000
10	СВЧ-печь	Braun	6	3000,0000

Таблица НаличиеТехники

Магазин	Техника	Количество
1	1	40
1	8	25
2	5	45
2	6	30
2	8	15
3	1	50
3	6	20
3	7	15
5	2	42
5	5	35
6	1	45
6	10	10
7	4	25
7	8	25
8	1	15
8	3	50
8	4	62
9	8	21
10	1	30
10	10	20

Вариант 18 Стоматологическая поликлиника

Необходимо хранить информацию о пациентах (номер пациента, фамилия, адрес, год рождения), о предоставляемых видах услуг (код, название), а также об оказании услуг пациентам с указанием стоимости и времени приёма.

1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.

1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех услугах из таблицы Услуги, кроме услуги с кодом 100. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.

1.2. Разработать триггер, который позволяет изменять значение стоимости только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.

2. Создание триггеров для представлений.

2.1. Создать представление с информацией об оказанных услугах. Представление должно включать столбцы номер пациента, фамилия, код, название, время и стоимость.

2.2. Разработать триггер, который позволяет вносить информацию об оказанных услугах через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем пациенте или услуге.

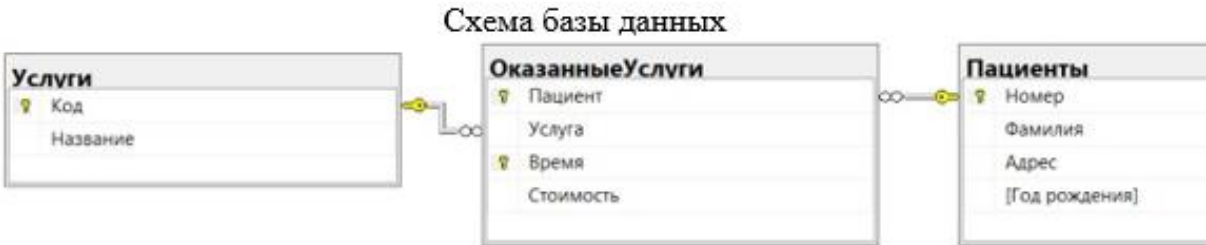


Таблица Пациенты

Номер	Фамилия	Адрес	Год рождения
1	Петров	Солнечная, 8, 46	1989
2	Иванов	Радищева, 22, 22	1961
3	Потапова	Горького, 37, 12	1968
4	Зотов	Павлова, 1, 10	1984
5	Ковалева	Свободы, 81, 70	1989
6	Сидоров	NULL	1989
7	Фролов	Почтовая, 65, 6	1961
8	Татарина	Соборная, 2, 10	1975
9	Ильин	Урицкого, 67, 3	1987
10	Сафронова	Каляева, 13, 20	1980

Таблица Услуги

Код	Название
100	Удаление зубов
101	Лечение зубов
102	Протезирование
103	Отбеливание
104	Чистка полости рта
105	Декоративное украшение зубов
106	Рентгенодиагностика
107	Пародонтология
108	Исправление прикуса
109	Реставрация зубов

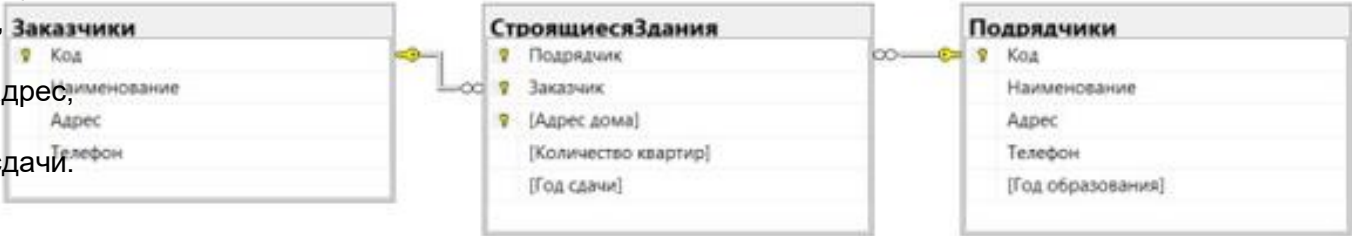
Таблица Оказанные Услуги

Пациент	Услуга	Время	Стоимость
1	102	12:00:00	600,0000
8	104	15:00:00	500,0000
8	109	10:00:00	100,0000
10	107	08:00:00	250,0000
6	104	17:00:00	1000,0000
2	105	21:00:00	750,0000
4	103	19:00:00	400,0000
7	102	12:00:00	5000,0000
3	109	11:30:00	260,0000
4	106	10:40:00	340,0000
1	102	17:10:00	560,0000
9	104	15:00:00	50,0000
10	107	08:45:00	100,0000
7	100	09:00:00	2500,0000
3	100	10:30:00	400,0000
2	103	11:00:00	980,0000
1	100	16:00:00	120,0000
4	101	12:40:00	300,0000
9	100	14:35:00	460,0000
6	105	20:00:00	900,0000

Вариант 19 Строительство жилья

Необходимо хранить информацию о существующих подрядчиках (код, название юридического лица, адрес, телефон, год образования фирмы), о заказчиках (код, название юридического лица, адрес, телефон), а также о строящихся зданиях с указанием адреса дома, кол-ва квартир и года сдачи.

Схема базы данных



- 1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.
- 1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех заказчиках из таблицы заказчики, кроме заказчика с кодом 21234581. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.
- 1.2. Разработать триггер, который позволяет изменять значение количества квартир только на меньшее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.
- 2. Создание триггеров для представлений.
- 2.1. Создать представление с информацией о строящихся зданиях. Представление должно включать столбцы код подрядчика, наименование подрядчика, код заказчика, наименование заказчика, адрес дома, количество квартир, год сдачи.
- 2.2. Разработать триггер, который позволяет вносить информацию о строящихся зданиях через представление. Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем подрядчике или заказчике.

Таблица Подрядчики

Код	Наименование	Адрес	Телефон	Год образования
602338546372	ОООСтройимпульс	ул. Земляной Вал, д.36	9163348	2000
602345673248	ТСКПрактика	Выборгское ш., д.38.	5163348	2000
602356789475	ОООРосСтрой	ул. Ленская, д.17.	5259201	2004
602394758654	АТМИнжиниринг	Чермянский пр-д, д.7.	7416321	2000
604367282764	СКСтройтэк	ул. Лесная, д.59.	5029962	2006
780456375834	ПКСтройка	ул. Совхозная д.9.	7559050	2002
802374829475	ЗАОСтимекса	ул. Титного, д.78.	2601983	2003
803225667225	СКСтрой	ул. Судакова, д.10.	1014102	2003
803267843988	ЗАОЮни-Строй	ул. Строителей, д.6.	8949485	2003
803483467899	СТДДевелопмент	ул. Тверская, д.12.	2095328	2007

Таблица СтроящиесяЗдания

Код	Наименование	Адрес	Телефон
21234581	ИП Захаров	г. Рязань, ул. Сенная, д.2.	983345
21234583	ИП Дмитров	г. Рязань, пл. Димитрова, д.8.	770597
21234584	ИП Трофимов	г. Рязань, ул.им. проф. Никулина, д.1.	455595
21234591	ИП Дёмин	NULL	567877
21234594	ИП Уткина	г. Рязань, ул.5-ая линия, д.2.	764263
31234587	ИП Еремеев	г. Москва, ул. Набережная, д.2,кор.3.	9123345
31334588	ИП Степнов	г. Москва, ул. Казакова, д.1.	9121723
41445697	ИП Бакунин	г. Санкт-Петербург, ул. Галерная, д.3,кор.3.	9561245
41445698	ИП Измайлов	г. Санкт-Петербург, ул. Чапыгина, д.23.	9562367
41445699	ИП Орлов	г. Санкт-Петербург, ул. Псковская, д.7, кор.1.	7782312

Подрядчик	Заказчик	Адрес дома	Количество квартир	Год сдачи
602338546372	31234587	г. Москва, ул. Восточная, д.35.	30	2015
602338546372	41445698	г. Санкт-Петербург, ул. Гонимая, д.27, кор.4.	105	2021
602345673248	21234591	г. Рязань, ул. Татарская, д.57.	50	2013
602345673248	41445698	г. Санкт-Петербург, ул. Асафьева, д.12.	55	2018
602356789475	21234584	г. Рязань, ул.Костычева, д.21, кор.2.	25	2019
602394758654	41445697	г. Санкт-Петербург, ул.Дульская, д.44, кор.3.	57	2015
604367282764	31334588	г. Москва, ул.Ватулина, д.48, кор.1.	45	2020
604367282764	41445697	г. Санкт-Петербург, ул.Белинского, д.39.	55	2013
780456375834	21234594	г. Рязань, ул.Затинная, д.15.	89	2022
780456375834	31234587	г. Москва, ул.Бакова, д.9.	40	2019
780456375834	31334588	г. Москва, ул.Алтайский, д.34.	21	2016
802374829475	31234587	г. Санкт-Петербург, ул.Миллионная, д.26.	25	2020
803225667225	21234583	г. Рязань, ул.Шедрина, д.14, кор.3.	25	2017
803225667225	21234591	г. Рязань, ул.Урицкого, д.8.	65	2014
803225667225	21234594	г. Рязань, ул.Свободы, д.24.	70	2020
803267843988	21234583	г. Рязань, ул.Гоголя, д.27.	62	2022
803267843988	21234591	г. Рязань, ул.Чернышевская, д.13.	50	2020
803267843988	41445698	г. Санкт-Петербург, ул.Борова, д.22.	12	2016
803483467899	21234581	г. Рязань, ул.Пушкина, д.65.	121	2022
803483467899	21234584	г. Рязань, ул.Костычева, д.21, кор.1.	157	2020

Вариант 20 Сотовые и интернет компании России

Необходимо хранить информацию об операторах сотовой и интернет связи (название, адрес центрального представительства, фамилия генерального директора, телефон), города (название города, название области, количество жителей), а также о количестве абонентов каждой сотовой компании в отдельных городах за последние 3 года.



1. Изучение оператора CREATE TRIGGER для создания триггеров к базовым таблицам БД.

1.1. Разработать AFTER-триггер, который позволяет удалять информацию обо всех операторах из таблицы операторы, кроме оператора с кодом 1. Считать, что пользователю предоставляется возможность удаления множества записей в таблице.

1.2. Разработать триггер, который позволяет изменять значение количества жителей только на большее значение. Считать, что пользователю предоставляется возможность модификации множества записей в таблице.

2. Создание триггеров для представлений.

2.1. Создать представление с информацией об абонентах. Представление должно включать столбцы код города, название, код оператора, название, количество абонентов, год

2.2. Разработать триггер, который позволяет вносить информацию об абонентах через представление.

Считать, что пользователю предоставляется возможность вставки только одной записи через представление. Предусмотреть вывод сообщения об ошибке в случае ввода информации о несуществующем городе или операторе.

Таблица Операторы

Код	Название	Адрес	Генеральный директор	Телефон
1	МТС	Москва, Марксистская, 4	Евтушенков Феликс	482649
2	Мегафон	Москва, Оружейный переулок, 41	Вермишян Геворг	568425
3	Билайн	Москва, Краснопролетарская, 4	Торбахов Александр	783613
4	Tele2	Москва, Ленинградское шоссе, 39А	Эмдин Сергей	173847
5	Сотком	Рязань, Есенина 47/24	Майзельс Игорь	407247
6	Ростелеком	Москва, Гончарова, 30	Иванов Сергей	604847
7	Дом.ру	Пермь, шоссе Космонавтов, 111	Кузьяев Андрей	NULL
8	Видикон	Рязань, Маяковского, 103/42	Кривошапов Александр	NULL
9	Тинькофф Мобайл	NULL	Хьюз Оливер	894026
10	Yota	NULL	Авдолян Альберт	493625

Таблица Города

Код	Название	Область	Количество жителей
1	Москва	Московская	12678079
2	Рязань	Рязанская	526919
3	Пермь	NULL	1055397
4	Коломна	Московская	144838
5	Луховицы	Московская	30443
6	Рыбное	Рязанская	20677
7	Самара	Самарская	1156659
8	Калуга	Калужская	328871
9	Скопин	Рязанская	27163
10	Ряжск	Рязанская	21189

Таблица Абоненты

Город	Оператор	Количество абонентов	Год
1	1	150000	2018
1	2	12000000	2019
1	9	10000	2020
2	2	95000	2019
2	5	30000	2020
2	8	45000	2020
3	6	120000	2018
4	2	145000	2019
4	4	10000	2020
5	7	30000	2020
5	9	18000	2019
6	3	5000	2019
7	1	500000	2020
7	2	478000	2020
7	6	200000	2019
7	7	349000	2019
8	1	1000	2019
8	5	524	2018
8	8	315	2020
9	8	514	2020